# PLC PROGRAMMING USING RSLOGIX 500

## ADVANCED PROGRAMMING CONCEPTS

GARY D. ANDERSON

# PLC Programming Using RSLogix 500

## Advanced Programming Concepts

# Book 2

By
Gary D. Anderson

PLC Programming using RSLogix 500
Advanced Programming Concepts (Book 2)



Every effort has been made in preparation of this book to ensure accuracy in presenting these concepts, information, and the typical usage of instructions when programming in the RS Logix 500 development platform. The information contained herein is sold without any expressed or implied warranty.

First Published: June 2015

PLC Programming using RSLogix 500: Advanced Programming Concepts

ISBN: 978-1-7341898-6-5

Contents

## Preface:

In my previous book, “Basic Concepts of PLC Programming”, I focused on many of the basic programming instructions, concepts, and their use in ladder logic programs. These instructions included the (XIC), the (XIO), and the (OTE) - including the (OTL) latching and (OTU) unlatch instructions. Also covered, were “timer” and “counter” instructions, the use of the “OSR” or “one-shot-rising” instruction, and the MCR instruction. These will also be shown by example and covered in more detail in this book, since they are the principle building blocks of any ladder logic routine.

In “Advanced Concepts”, we will look into the use of many other “specialized” instructions, such as the (RTO) “retentive timer”, the use of the (RES) “reset” instruction, and “counters”, which are also utilized in a wide array of applications. Also in the following chapters, we will discuss “register type” instructions that accomplish functions such as data comparisons or manipulation, by using entire “words” of data, either 16 or 32 bits. I’ll also cover the basic math instructions that can be used in “scaling” for analog values, and some discussion on instructions that provide program flow to subroutines.

Hopefully by this time, you’ve reached some degree of a “comfort zone” in addressing the instructions we talked about in the “Basics” book. In this second book I’ll not spend much time covering those topics again unless it is to focus on some variation not covered by the “syntax” chart included in the first book.

The focus for this volume should accomplish the following objectives:

- To understand the use of “RSLinx” drivers and basic communication protocols.

- To understand fully the use of “timers” – the TON, TOF, RTO and the use of the reset (RES) instructions.

- To understand the use of “counters” – the CTU and CTD, and the way in which “counters” and “timers” are often “nested” together to develop a specific time-base for scheduled maintenance or other process related issues.

- To understand the use of register instructions such as the MOV, CLR, COP and FLL.

- To understand the use of the math instructions: ADD, SUB, MUL, DIV & DDIV.

- To understand comparison and program flow instructions.

- To better understand analog scaling, what it means, and different ways of accomplishing scaling within your ladder logic program including the use of the “math” instructions, the SCL, and the SCP instructions.

- One thing I have decided to include, and in fact begin in the following chapter, is a brief discussion with examples on using RSLinx and its related communication protocols, drivers and physical media. Communication between your interface devices, such as a laptop, can sometimes be a

troublesome issue. So I think it’s important to address the whole issue of connectivity at the beginning of our programming experience. I’ve also included the “driver & cable” chart that was shown in book 1, simply because it’s a good reference to have handy, and has worked well for me.

# RSLinx & SLC 500 Communications

The SLC 500 family of processors is basically composed of the following models: the 5/01, 5/02, 5/03, 5/04 and the 5/05, although I seldom see 5/01's or /02's anymore. Still, the 5/03's and newer are still very much in use, and in my opinion, a good programming tool for many applications. Each of these employ one or two different "channels" where the interface device cable may be connected. The communication channels are slightly different from model-to-model so it's necessary understand the different Allen-Bradley communication protocols used for each model of processor. The 5/01 and 5/02 had only a single communication channel, while the 5/03, 5/04, and 5/05 have two channels designated channel 0 and channel 1.

Following are the basic Rockwell Automation /Allen-Bradley communication protocols, and a brief description of each. Then we can consider the channels and their default settings for these five different processors. As you will see, these same communication protocols are used with other Allen Bradley processors, such as the PLC-5 models, the Micrologix processors, and also the ControlLogix and CompactLogix processors which use the RSLogix 5000 application software.

Understanding the difference between a "protocol", and the language that surrounds the descriptions of "physical media" wiring and connection types, can at times be confusing because many of the terms are used interchangeably. For example, someone may simply say that they are using a RS-232 driver, but technically this refers to the physical media rather than the drivers or protocol – which would be the DF-1. Therefore, it's a good idea to at least define what a protocol is, because it may work with

several different types of network connections and wiring. A specific processor channel may work with several different drivers and configurations as well.

## Protocols:

A communications protocol is simply a set of established rules for exchanging data between computers or other electronic devices, such as a PLC processor. These rules define details pertaining to the syntax used, data and addressing formats, types of flow control, sequencing and size of the data frame, error detection, acknowledgements, time-out periods, and many other important details.

Physical characteristics of networks, on the other hand, will be designated as RS-232, RS-422, RS-485 or Ethernet. Perhaps more importantly, these network types will be defined by those standards which specify different physical characteristics in terms of voltage, wiring, the number of “nodes” or devices that can be accommodated, the length of cable runs, as well as the speed and throughput of data transmission.

To illustrate the physical context of the RS-232 standard (EIA/TIA-232); it defines the voltage levels for what will be interpreted as a logic “1” and “0” during data transmission. The specified voltage range for a logic “1” is a voltage pulse between negative (-3) and negative (-25) volts; a logic “0” voltage is between (+3) and (+25) volts. Protocols are designed to work with each of the physical characteristics defined by these standards. The channel connections Allen Bradley builds into their processors are configured to work with the following protocols:

## DF-1 Protocol:

This protocol supports the EIA/TIA-232 standard. For this reason it is usually thought synonymous with RS-232 and serial communication, and supports data transmission rates of up to 19.2K baud, or 19,200 bits/s. It supports “full-duplex” peer-to-peer communication, sometimes referred to as “point-to-point” where two devices can communicate simultaneously, and also “half-duplex” communication where devices take turns sending and receiving data in a master / slave type of network.

## DH-485 Protocol:

This is Allen Bradley's "Data Highway" protocol, and is used for RS-485 communication (EIA/TIA-485). Once again; just as with RS-232, the RS-485 standard defines the physical characteristics detailing wiring media and other electrical properties.

The DH-485 protocol is designed for use with the RS-485 standard. A DH-485 network is a LAN (local area network), designed for multi-point connections, and used in industrial applications where electrical noise may be a problem. These networks allow connection of up to 32 nodes or devices, such as PLC controllers, HMI's and personal computers. As such, this protocol uses a multi-master, token-passing scheme which allows all the devices on the DH-485 network to have a turn in sending and receiving data. This protocol supports baud rates of up to 19.2K using Beldon 9842 or 3106A cable with RS-485 specifications.

## DH Plus (DH+) Protocol:

The DH+ protocol is used with the PLC-5 family of Allen Bradley processors and also the SLC 500 model 5/04. It is similar to the DH-485 with the exception that it will support up to 64 nodes or devices per network and communication baud rates increase to 57K, 115.2K and 230.4 K. Physical media is Beldon 9463 wiring sometimes referred to as “blue hose” wiring.

## Ethernet & Ethernet/IP Protocol:

These use standard TCP/IP protocol common to industrial networks, operates at a data rate of 10 Mbps (10 Base-T), and can support a much larger number of individual devices than the DH-485 or DH+ networks.

Once again, the SLC 500 processors are set up to operate with these “default” protocols and are equipped with the “physical” connection point to accommodate these types of networks. Here is a listing of the SLC 500 processors and their default channel configurations.

| Processor Type | | Physical Channels & Communication Protocols | | | |
|---|---|---|---|---|---|
| | | RS-232 / EIA-232 | RS-485 / EIA-485 | DH+ | Ethernet |
| SLC 5/01 | | | DH-485 Protocol | | |
| SLC 5/02 | | | DH-485 Protocol | | |
| SLC 5/03 | Channel 0 | DF1 Full and Half-Duplex, DH-485, and ASCII Protocols | | | |
| | Channel 1 | | DH-485 Protocol | | |
| SLC 5/04 | Channel 0 | DF1 Full and Half-Duplex, DH-485, and ASCII Protocols | | | |
| | Channel 1 | | | DH+ Protocol | |
| SLC 5/05 | Channel 0 | DF1 Full and Half-Duplex, DH-485, and ASCII Protocols | | | |
| | Channel 1 | | | | Ethernet TCP/IP Protocol |

Now we will look into the steps of starting RSLinx, and selecting the appropriate protocol and configuration for communication with a SLC-500 processor.

Generally; I first start RSLinx and establish a communication link, then start the RSLogix 500 application software and go “online”. However, you can often do this

directly from the RSLogix application software. It will start up RSLinx and allow you to choose the network processor, or the PLC if you are directly connected for point-to-point communication. Starting the RSLinx application first, is simply how I was taught, and so I continue to use the same steps today whenever connecting to an Allen Bradley PLC or PanelView. It allows you to know quickly if communication can be established with a processor; and so for SLC-500, PLC-5's, Micrologix, ControlLogix, CompactLogix, Panel Builder 32 and Factory Talk applications, I still start-up and link with RSLinx before starting the application program.

At times a driver has been left in the configuration menu when RSLinx was "closed", I've found it is usually best to "stop and delete" existing drivers from the configuration menu - and start from a "clean slate". This tends to be an advantage in avoiding port conflicts and such.

Also note that since most laptop computers no longer include a built-in serial port, it is often necessary to use one of the USB ports that are now the standard. The USB adapters listed in the following chart work well for DF-1, DH-485, and DH+ applications. They require drivers to be installed which are included with the RSLinx software, as well as the installation disks that come with the adapters.

The following *Cable & Adapter Chart* is a listing of the cables and the adapters which I routinely use to communicate with different Rockwell Automation products. I stopped using the PCMK cards because it seemed that the cards, software, or cables would always be changing, and they were fragile at the cable connection point into the laptop. Therefore the PCMK category is not shown on this listing. I just believe these are better options.

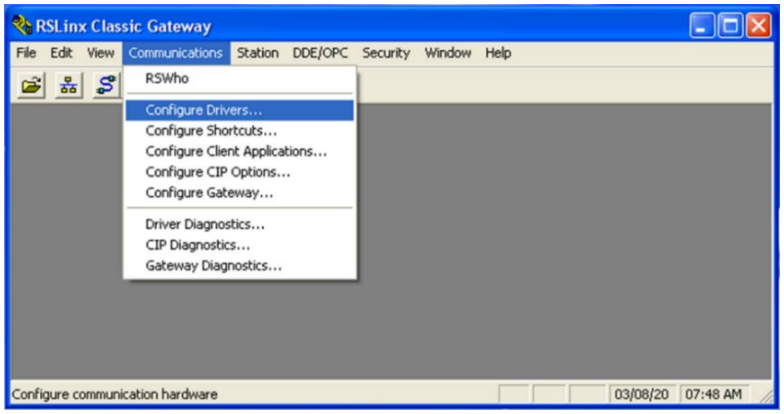
| Controllers, Adapters & Interface Cables | | | RS Linx Settings | | | |
|---|---|---|---|---|---|---|
| Controller | Adapter & Port | Cable | Driver Type | Driver Name | Device Type | Notes |
| SLC 5/01, 5/02, 5/03 Controllers (DH-485) | 1747-UIC / RS-485 port | 1747-C13 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| SLC 5/03, 5/04, 5/05 Controllers (DH-485) | 1747-UIC / RS-232 port | 1747-CP3, 1756-CP3 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select the correct COM port / CRC / 19.2 Kbps |
| SLC 5/04 and PLC-5 Controllers (DH+) | 1784-U2DHP | Integral 8-pin Din | RS-232 or 1784-U2DHP | DF1 | 1770-KF2 / 1785-KE/SCANport | Select COM port / use CRC / and node |
| MicroLogix Controller (DH-485) | 1747-UIC / RS-232 port | 1761-PM02 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| PanelView 300 or Higher (DH-485) | 1747-UIC / RS-485 port | 1747-C13 or 2711-NC13 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| 1747-AIC (DH-485) | 1747-UIC / RS-485 port | 1747-C13 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| 1747-AIC (DH-485) | 1747-UIC / RS-232 port | 1747-CP3, 1756-CP3, 1761-PM02 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| CompactLogix Controller (DH-485) | 1747-UIC / RS-232 port | 1747-CP3, 1756-CP3 | RS-232 | DF1-1 | 1770-KF3 / 1747-KE | Select COM port / use CRC / 19.2 Kbps |
| ControlLogix, CompactLogix | RJ45 / Straight for Switch connection, Crossover cable for peer-to-peer | | Ethernet or Ethernet/iP | | | Configure IP address of interface device |
| PanelView Plus | RJ45 / Straight for Switch connection, Crossover cable for peer-to-peer | | Ethernet or Ethernet/iP | | | Configure IP address of interface device |

## Step-by-Step Configuration:

Here are specific steps for connecting with a SLC-500 after starting RSLinx. Shown below, is the initial RSLinx start-up screen showing the dropdown from the Communications Tab. If you are using a desktop computer in a centralized location, and have already established communications with a network, you could select the “RSWho” option. The RSWho option will populate a dropdown organizer window showing all the processors on your network. At this point, you could simply select the individual processor from this screen.

However; in most of the real-world scenarios where I’ve needed to go-online with a PLC, this has not been the situation. It is usually a single controller, perhaps a part of a small localized network having remote I/O and HMI’s, and used for a specific process or machine. In this case; proceed to and through the following steps which outline setting-up simple point-to-point communication.

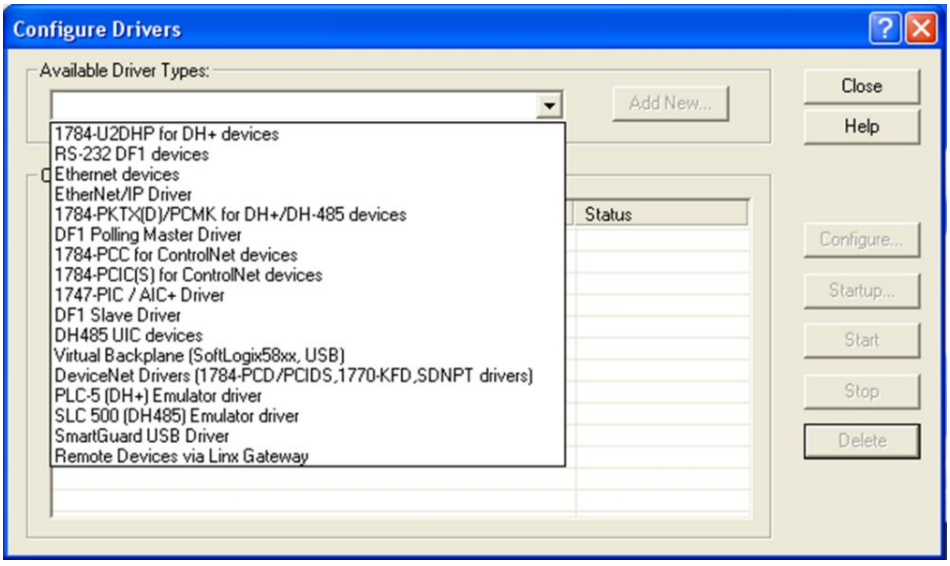
**Step 1**: Pick and configure the appropriate driver using the “Communications” Tab located on the top menu bar. Select the Configure Drivers option:



**Step 2**: Select from the various driver “types” for the connected communication channel used by your processor. If connecting to a 5/04 processor, you would select either the 1784-U2DHP or the RS-232 DF1 driver; for a 5/05 you might use the Ethernet device driver but could also use the RS-232 DF1; if connecting to a ControlLogix platform, the EtherNet/IP driver is an option. Also shown are the options for DeviceNet and ControlNet networks.

Once again, use the above chart, and become familiar with the default channels for different controller models. For reference: See Rockwell Pub 1747-2-39 which covers the SLC-500 family of processors.

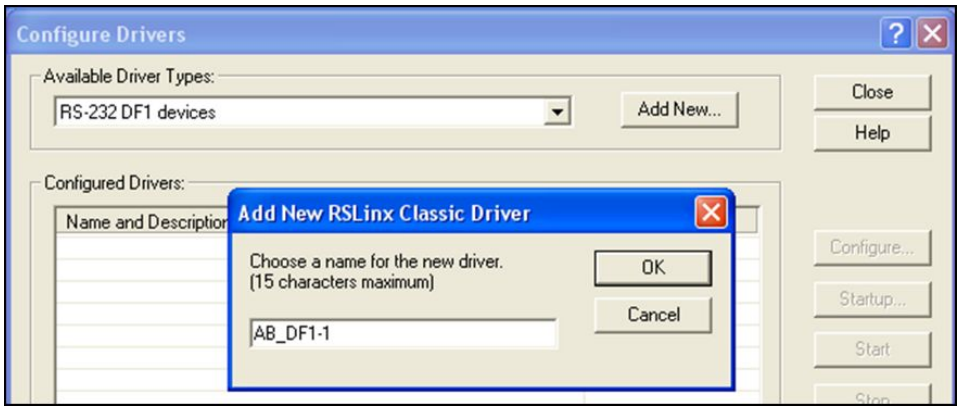
For our example; choose the RS-232 DF1 devices.



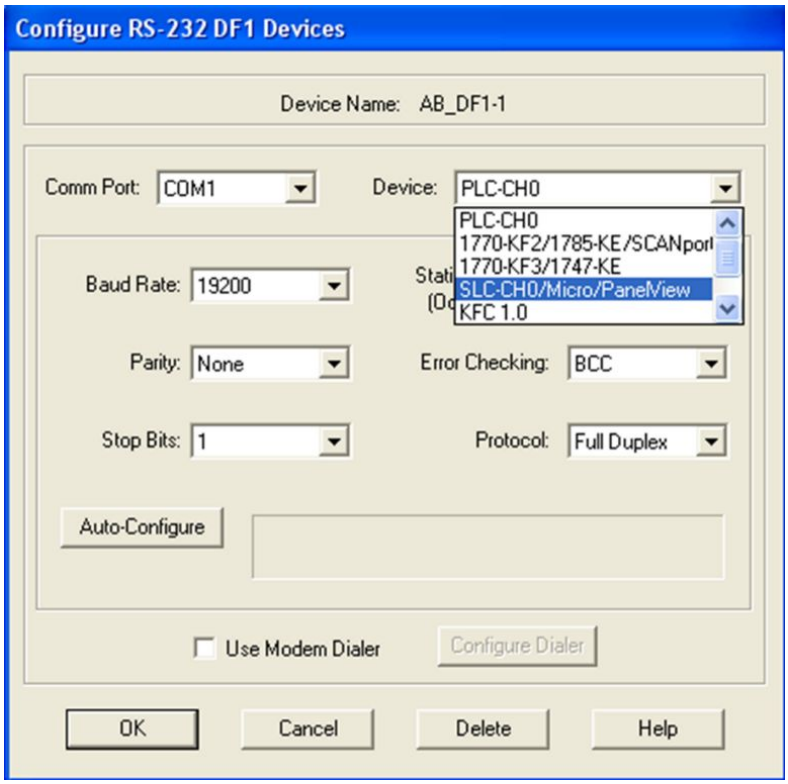
Note that once a driver type is selected, it will often bring up other options corresponding to the chart shown on page 8. For instance; when selecting RS-232 DF1 devices, you will be presented with several other options in the “Device” dropdown window. These would include options for PanelView, MicroLogix, or the USB adapters listed in the chart.

After making this selection click on the “Add New” button and proceed to step 3.

**Step 3**: Click “OK” for the default driver name, and fill in the other applicable details to configure the driver.




Even if you plan on using the Auto-Configure option, as shown on the next page, it is usually important to first confirm the Comm Port being used on your PC or laptop. This would be necessary; in any case, if using a USB adapter with a CP3 cable connection; or using the 1770-KF2/1785-KE/SCANport or 1770-KF3/1747-KE adapters shown in the Device dropdown menu.



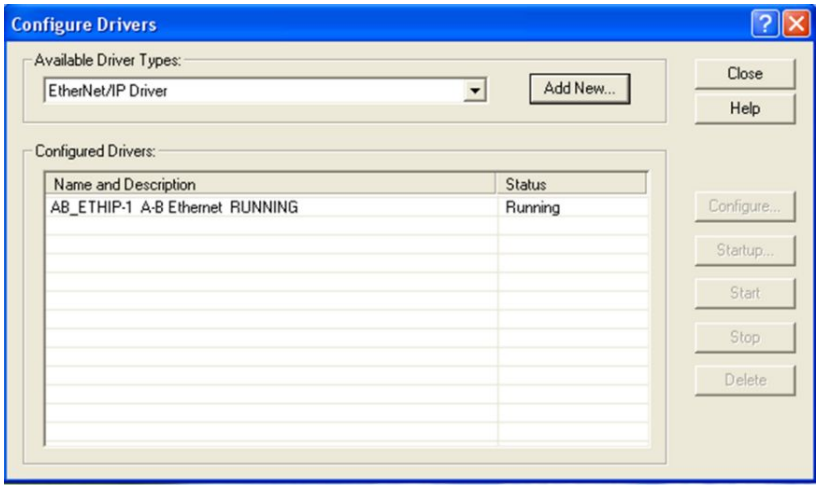

If you are using a serial cable, such as Allen Bradley's CP3 or the PM02 cable directly from a DB-9 or USB adapter on your laptop, you should be able to simply select the Auto Configure feature. The software will scan for the connection and fill in all the other details that pertain to baud rate, parity, error checking and so forth. You will see this selection button toward the lower left side of the configuration box. When the fields are filled and you get the "Auto-configuration successful" message, then click the "OK" button and proceed to the next step.


Note on Step 3:

If you are using one of the USB adapters such as the 1747-UIC or the 1784-U2DHP, then you cannot use the auto configure feature, you must manually select the Comm Port, set Error Checking to CRC, and possibly Baud Rate and Parity. The USB adapters will often connect through Comm Ports other than Comm 1, 2, or 3, and therefore must be manually entered into the configuration menu, usually along with the type of error checking (CRC), device type and station node. The small manual that comes with these adapters is easy to follow regarding these manual entries. Now it's time to see if we are communicating with the PLC.

**Step 4**: After configuring and clicking the [OK] button, RSLinx will let you know your driver is "running". At this point, you can just [Close] this menu and go back to the

Communications window that will show you if you have an established connection with the PLC.

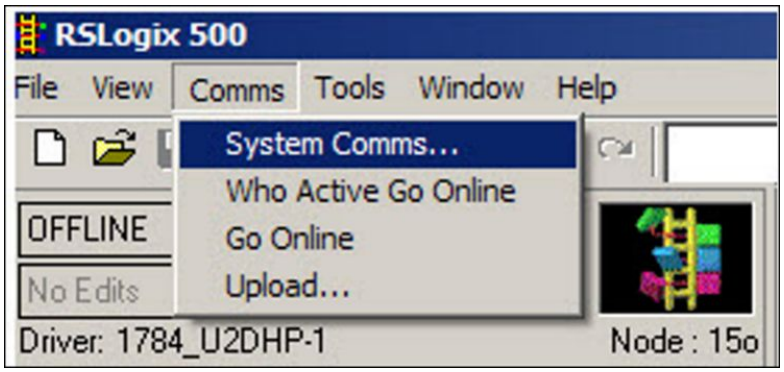



Remember that this is an example of point-to-point communication, where only one processor is shown. If other items are on the same local network, such as an HMI, these might be shown as well. Whatever is shown in this network window are items with which RSLinx has an established communications path. In any case, you simply select the PLC processor with which you want to communicate and highlight its symbol, as in the following example.

After highlighting the processor, in this case a SLC-5/04, you can “minimize” the RSLinx program and just leave it running in the background. At this point we are ready to open our application software, RSLogix 500, and go online with our processor.

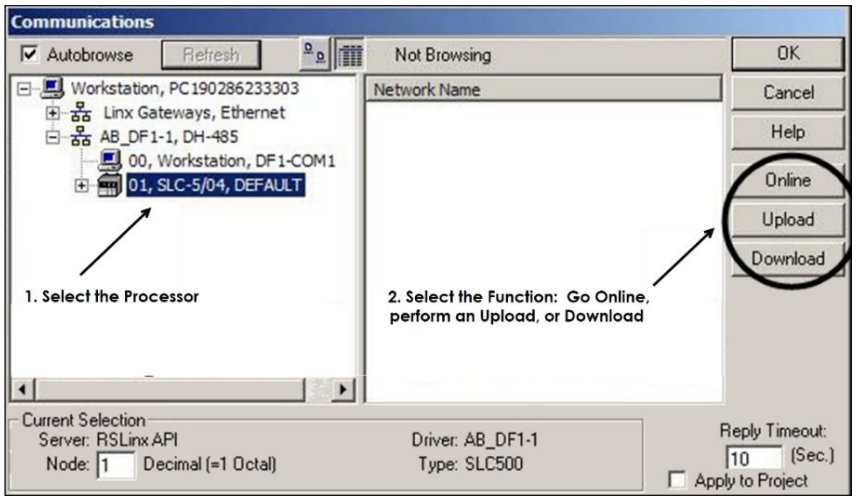
**Step 5**: Open RSLogix 500

**Step 6**: Select the *Comms Tab* on top bar to select your processor, and then choose from the four different options:




You can go online if you want to monitor a running program, or make online edits to a running program; you also have options to upload a program from the connected PLC, or to download a program into the connected PLC.

It is often necessary to first select the System Comms option and then choose, similar to what you did while in RSLinx, the processor with which you want to communicate. Once this is done, you should see the three options: select from the Online, Upload or Download options shown below.




Note: Connecting to Existing Equipment:

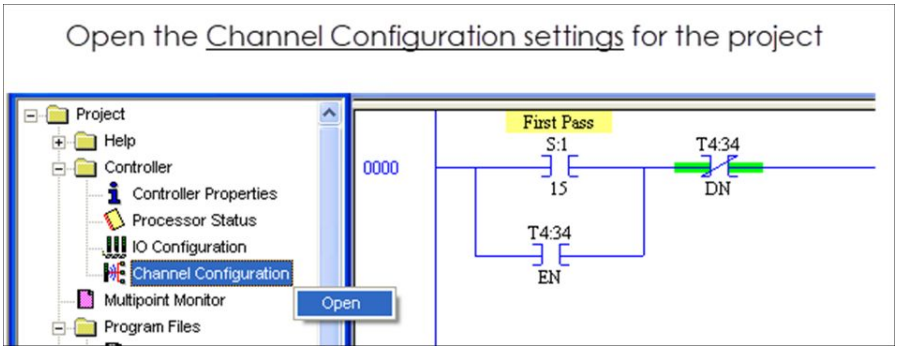
If you are attempting to connect to existing equipment but having difficulty, you may need to check the comm channel settings of the program loaded into that particular processor, as they may be different from the default settings of the processor model. This is an option if your programs are backed-up to a server or some other type of storage media, where they can be opened and viewed offline with the RSLogix 500 software.

I recently tried to go online with a SLC 5/04 using a CP3 cable, directly from my laptop. I attempted using the auto-configuration option, but it would never communicate successfully. I had an older copy of the program, and when I checked the channel configuration it turned out that channel 0 had been changed from the default DF-1 to the DH-485 type which requires the DH (Data Highway) set-up and drivers. I switched to the 1747-UIC adapter, which uses a USB port and the same CP3 null-modem cable, and was able to easily connect and communicate with the processor. So, just because the default channel configurations for a new “out-of-the-box” processor are set a certain way, keep in mind that an existing PLC may be configured differently – it

all depends on the project configuration and program that has been loaded into the processor.

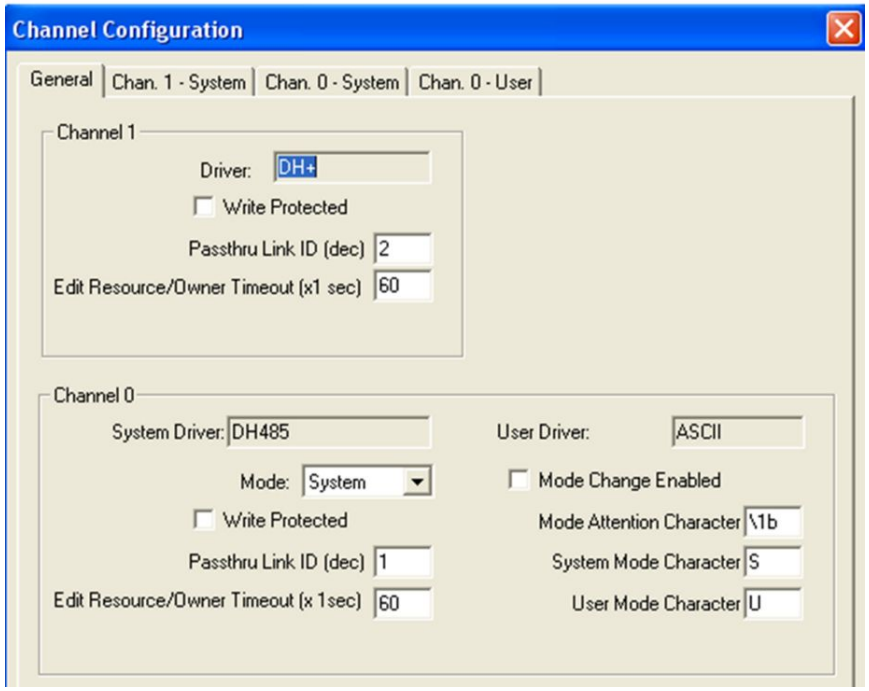
Channel 0 on the SLC 5/03, 5/04, and 5/05 models can be configured as DF-1 full or half duplex communication, as a DH-485 channel, or as using ASCII protocols. Note that Channel 1 for these processors is dedicated to DH-485 for the 5/03, DH+ for the 5/04, and to Ethernet TCP/IP protocol for the 5/05 processor.

Here are some examples of opening the Channel Configuration settings for a SLC-500 project:




The initial menu that opens for the Channel Configuration settings, shows the current settings for both channels of the processor. As you can see in this example; attempting to use the DF1 driver / RS-232 connection on Channel 0 was not going to work.

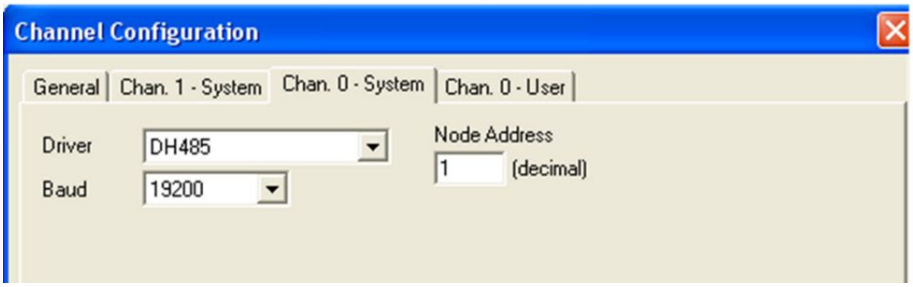
Current Configuration settings for
Channel 1 and Channel 0



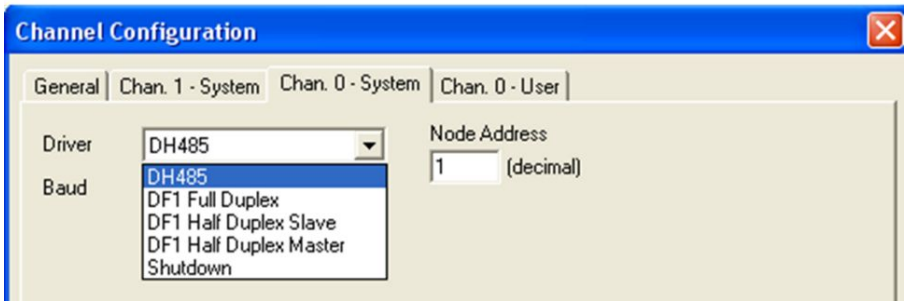

By selecting the Chan 0–System tab, you are given the option to change communication settings for the channel – if you wish to do so. In my scenario, I didn't change any processor settings, only the settings in RSLinx and the adapter cable I was using.

Here are two more screen captures which show the current settings of Channel 0, and the options available for the channel should you wish to change the configuration. There might be several reasons for making such a change; such as needing to accommodate a device on the other channel – like an HMI, or needing to configure for communicating with an ASCII terminal device. At any rate, here is where those changes would need to be made – just remember that you still need to be able to download any offline changes into the existing processor.

Channel 0 - System Tab




Options for Channel 0




## Hands-On-Exercise:

1. Set-up a SLC-500 or MicroLogix project, perhaps on a test bench, and go through the basic configuration steps for RSLinx shown in steps 1 – 6 for communication with a controller.

2. Try to establish a comm link using both channels of a processor.

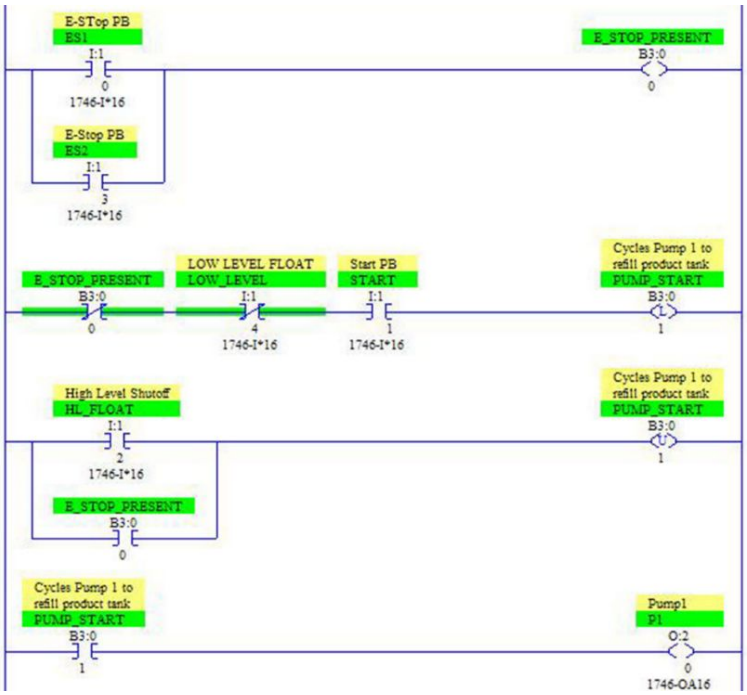
3. Write a basic program and download into this processor.

4. Upload a program, make some changes and download it back into the controller.

## Specialized “Action” Instructions

The OTL & OTU:

The OTL instruction operates in much the same way as an output energize or OTE instruction, in that it sets its bit location to “1” whenever the logic instructions that precede it become “true”. It differs however, in that it remains “on” even after its rung conditions become “false”. The addressed bit is turned “off” by the use of a corresponding “unlatch” instruction – the OTU. The OTU “unlatch” works very much the same in all aspects, except that when its rung conditions become “true”, it sets its addressed bit location to “0”, and turning the memory location “off”, or “false”.



These instructions are often used to implement different types of “holding” circuit logic, as shown in the example routine. In this scenario:

- If no E-Stops are present, and the low level condition is “true”, then a startup of Pump1 can be initiated by an operator pressing a momentary contact “Start” button. As evident on rung 2, the conditions of having No E-Stop Present and the Low Level both “true”, represent the initial conditions that must be present for the operator to start refilling the product container or tank to an acceptable level.

- Once started, the Pump1 output is “latched in” through the B3:0/1 bit. This bit is latched “on” even though the Start PB is momentary, and the rung logic becomes “false” when the pushbutton is released. In addition; the Low Level Float will soon change states as liquid level rises, making rung logic false for the (OTL) instruction.

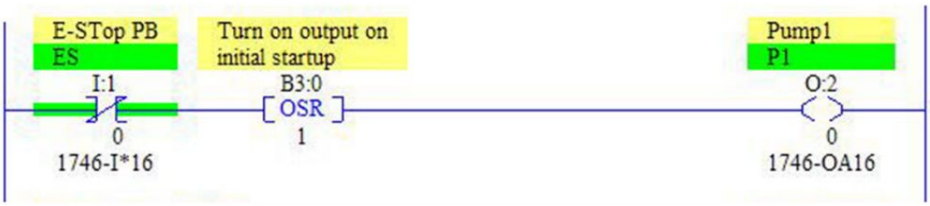
- When the High Level float actuates and I:1/2 becomes “true”, or if an E-Stop is actuated, then the (OTU) B3:0/1 bit is “unlatched” and Pump1 is turned off.

Note that the OTL and OTU can be used as an output module point or, as in our example, a bit from the B3 binary data file. Also note that in most cases, these latching instructions are “retentive” and thus will retain the current state of memory should power be lost - as long as the processor has a good battery.

## The OSR Instruction:

The OSR is an input instruction that triggers the occurrence of an action or event to happen only one time. Use the OSR when an event must start based on the change of state of the rung from a “false-to-true” status.

When the logic preceding the OSR goes from “false-to-true”, the OSR goes “true” for one scan. After the scan is complete the OSR becomes “false”, even though rung conditions that precede it may remain true. It will only become true again if preceding rung conditions transition, once again from false-to-true.




When addressing an OSR instruction, use a bit address from either the “B3” data file or a bit from an “integer” file such as the “N7” data file. The OSR uses this bit address to retain in memory its “last” state. Don’t address the OSR using discreet bits from either the input or output file tables, which are used by I/O modules.

## The RTO Instruction:

The RTO is a “retentive” on-delay timer which functions exactly the same as a TON, except that it “retains” its accumulated value whenever the rung transitions to “false”, the processor is switched from “run” to “program”, or even if power is lost. When the rung transitions back from “false-to-true”, the RTO simply begins timing and adding to the existing accumulated value. Just like the TON timer, the DN, TT, and EN bits are “useable” for the addressed RTO.

Here is the bit function for the RTO or “retentive” timer, as you can see, no real difference from the TON bit function.

| Bit Designated | Is set ("1") when: | And remains set until one of the following conditions: |
|---|---|---|
| Timer Done Bit (DN) Bit 13 | Accumulated value is equal to or greater that the Preset Value! | The "RESET" for the RTO is enabled. |
| Timer Timing Bit (TT) Bit 14 | Rung conditions are "true" and the ACC value is less than the Preset! | Rung conditions go "false" OR the DN bit is set! |
| Timer Enable Bit (EN) Bit 15 | Rung Conditions are "true"! | Rung conditions are "false" OR the RTO is reset! |

Note that the RTO Accumulated value is only reset back to its original value by the use of the reset instruction RES, having the same address as the RTO. This is also true of the CTU and CTD counter instructions. I will show the RES instruction and how these types of timer and counter combinations can be used together – “cascaded” or “nested”, to build longer measured time periods for scheduled maintenance or other production issues.

## The CTU & CTD Instructions:

Counters, aside from timers, are some of the most frequently used instructions in ladder logic programming. Counters operate in a very similar way to the RTO timer in that the accumulated value and status of the “control bits” are retained when the preceding rung conditions transition from “true-to-false”. Counters, like timers, must be addressed, and given a Preset value (set the final count). You can set the Accumulator value if desired – setting where the counter will begin its count, but this may also be left at a zero value. Three registers are assigned to a counter to hold the “Preset”, “Accumulator”, and “control” bit values.

| Counter Control Bits & 3-Word Allocation | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
| CU | CD | DN | OV | UN | UA | | | | *internal use-not addressable!* | | | | | | | *Status Word 0* |
| Preset Value (PRE) | | | | | | | | | | | | | | | | *Preset Word 1* |
| Accumulator Value (ACC) | | | | | | | | | | | | | | | | *Accumulator Word 2* |

| Addressable Bits | | Addressable Words |
|---|---|---|
| CU, "Count Up Enable" | (Bit 15) | PRE, "the Preset Value" |
| CD, "Count Down Enable" | (Bit 14) | ACC, "the Accumlated Value" |
| DN, "Done" | (Bit 13) | |
| OV, "Overflow" | (Bit 12) | |
| UN, "Underflow" | (Bit 11) | |
| UA, "Update ACC" | (Bit 10) | |

Here is a breakdown of the control status bits which can be used within your program.

- The CU “count up enable” and CD “count down enable” bits, 15 and 14 respectively, are set when the rung conditions preceding the CTU or CTD become “true”.

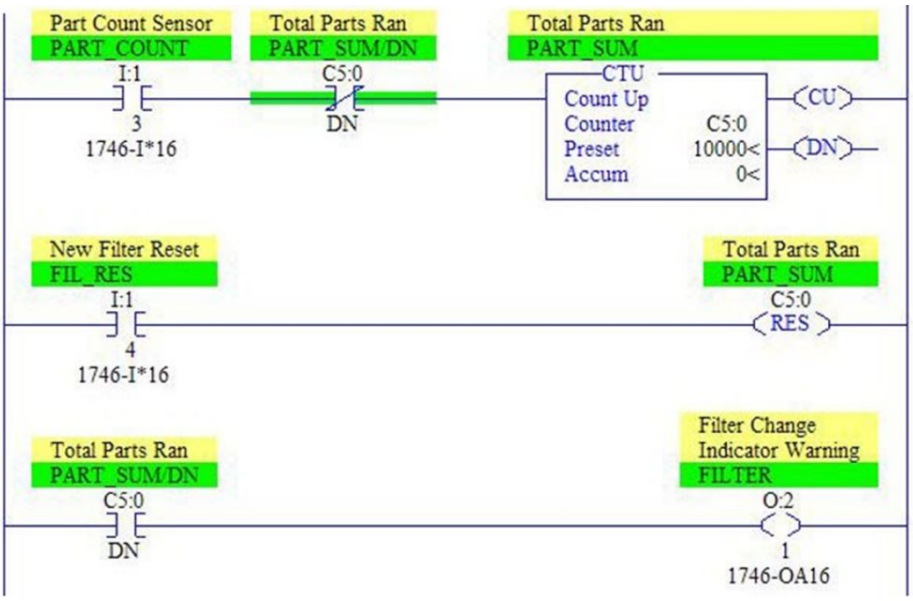
- The DN bit, number 13, is set to “1” when the counter’s accumulated value is equal to its preset value.

- The OV or “overflow” bit – number 12, is set when the CTU counter’s accumulated value increments over the maximum positive value.

- The UN or “underflow” bit – number 11, is set when the CTD counter’s accumulated value goes over from the maximum negative value.

- The UA or “update ACC”, bit 10, is set when the accumulated value has changed.

Here are a few rungs of logic using a CTU instruction and corresponding RES that might be integrated into a program to do something as simple as keeping a count on the number of parts that travel, via conveyor, past a sensor on input I:1/3.

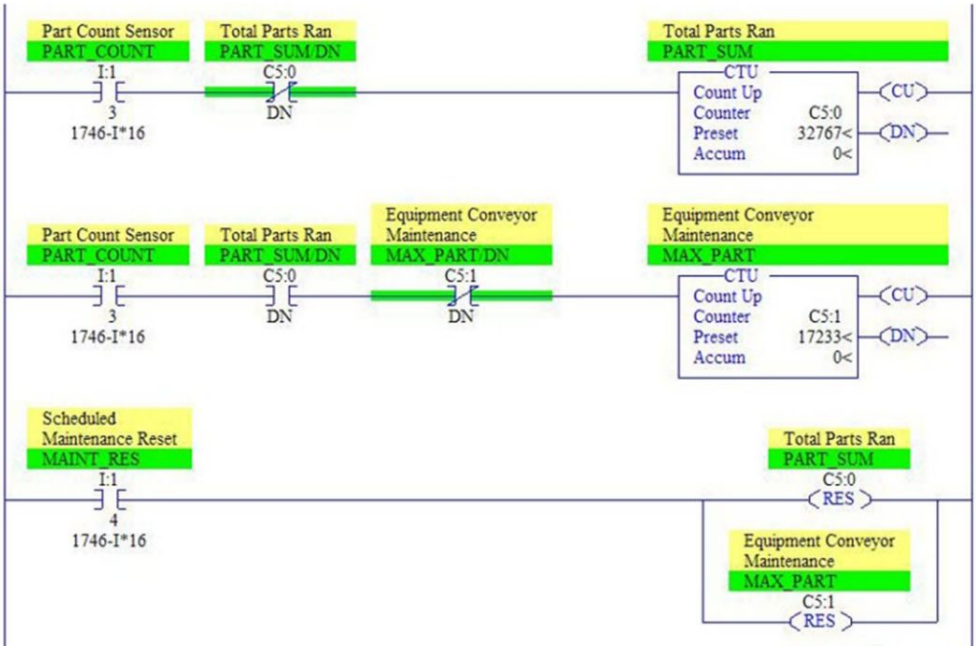



Once the “Accumulator value” is equal to the “Preset value”, the counter’s DN bit is set and the counter stops counting up - even though parts may still be moving past the I:1/3 sensor. The C5:0/DN bit will also set the warning flag on rung three, and will stay in this state until reset via the I:1/4 input. This could be a pushbutton or key-switch actuation, after a filter change or some other necessary task. Also the C5:0/DN bit could have been used to initiate multiple actions including stopping the equipment altogether until filters are changed.

## Cascading Timers & Counters:

Timers and counters can be “nested” together in a program to accommodate longer time periods. This could be, as in the above example, the program logic to indicate the need for a filter change after a specified number of parts produced, or any number of other scheduled maintenance issues. Here are two examples of how counters and timers might be combined to provide for these types of scenarios.

This first example indicates the need for filter changes after 50,000 parts have traveled past our sensor.




Since the “Preset” value is a signed integer value inhabiting one 16 bit register, the largest number we can count up to is 32767, remember bit 15 is used for the positive or negative sign and bits 0 – 14 for the number. Since we need to count up to 50,000, we can add a second

counter and make it start counting when the first one reaches its maximum value by using the C5:0/DN bit on the 2nd rung.

The second counter, C5:1 counts the additional 17233 parts until the total is equal to the 50,000 parts that signals the need for the scheduled maintenance procedure.

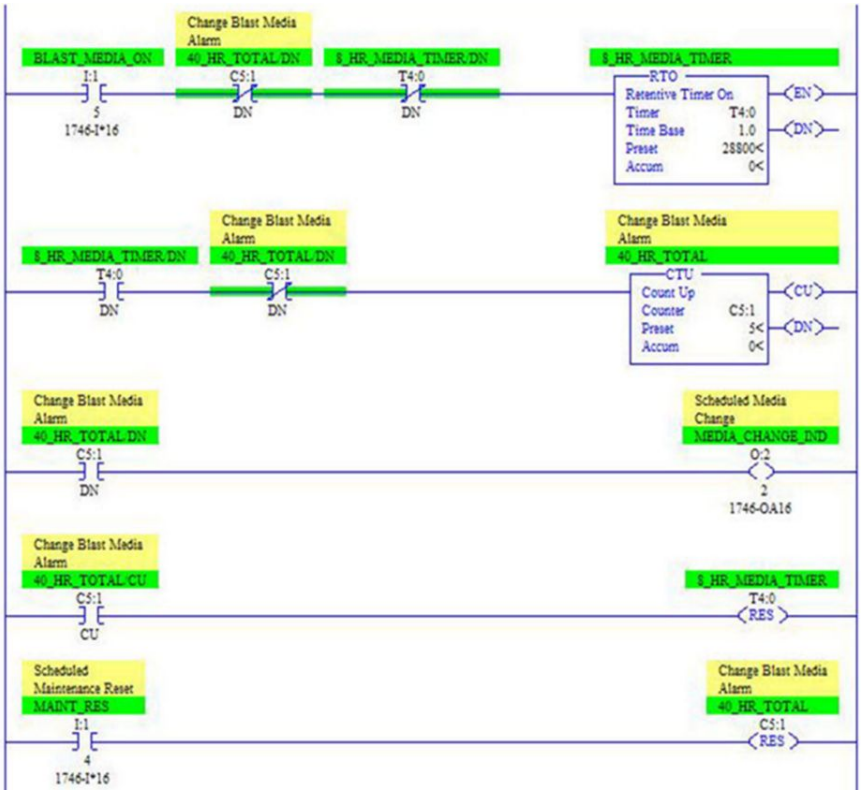
The C5:1/DN bit could also be used to shut the conveyor down, show an alert message or flag on an HMI, or in any number of ways alert the operator to the condition requiring maintenance.

In this example, both C5:0 and C5:1 are reset at the same time when the operator does the manual reset via I:1/4 input.

## Using the RTO:

In the following programming example; a scenario existed in which blast media used in a shot-peening operation, was being used longer than the prescribed 40 hour time period. By putting a few timer instructions into the existing program structure, and wiring an alarm from an output to the operator control panel – this condition was corrected. No HMI was in use so an indicator light was installed in the control panel. This alarm would alert management and the operator of the need to change blast media, and also to allow the reset after the media was changed.

Remember that the RTO/ DN bit remains set after its Accumulated value reaches the Preset value. It isn’t reset until the reset instruction is initiated by a new count on C5:1, shown on the fourth rung.




The RTO with a 1 second time base, increments to 28800 seconds or 8 hours, then pulses the counter C5:1 with a single count to the accumulated value. When the CTU reaches it preset value of 5, the C5:1/DN is set, the output indicating the need for media change is turned on. Note that the 8 hour RTO is reset whenever counter C5:1 is pulsed “true” by using the C5:1/CU bit. At this point the RTO is ready to begin the timing for the next 8 hour period of use.

The CTU is only “reset” by the use of the Maint_Res input, which could be a key switch or simple pushbutton.

Ideally, another protective condition should be placed into this rung of program logic. An XIC addressed to the C 5:1/DN bit, if put in series, or as an “AND” condition with the Maint_Res input switch, would prevent a random reset of the counter. It would only allow a “reset” if the counter had

accumulated to its full preset value – the full 40 hour time allowance.

## Basic Math Instructions

At face value, using math instructions are pretty straightforward and relatively easy to understand. Of course it is “how” you put these instructions together to obtain results that make these instructions so useful. The use of an ADD, SUB, MUL or DIV as separate instructions may be quite simple, but if the algorithm you need to calculate is more complex then keep in mind that the “order of operations” still apply - just as you would use in working an algebraic equation. For this reason I have found that symbols and descriptions are extremely helpful when added to these instructions. Good descriptions may save you significant time when trying to figure out if instructions are being used as part of a “larger” calculation.

Here is a chart describing these basic math instructions.

| Math Instructions | Descriptions |
|---|---|
| ADD | Add Source A to Source B and places the answer in the address you designate. |
| SUB | Subtracts Source B from Source A and places the answer in the address you designate. |
| MUL | Multiply Source A by Source B and places the answer in the address you designate. |
| DIV | Divides Source A by Source B and places the answer in the address you designate and the "math register". |
| DDV | Divides the "math register" by Source and places result in the designated address and "math register". |
| CLR | Clears a destination address and sets all bits of a word to zero. |
| SQR | Calculates the "square root" of a Source and places the result in a designated destination address. |
| SCL | Multiplies a Source by a rate and adds an offset value then places result in a designated address. |
| SCP | Produces "scaled" output values that have a linear relationship to input values. |
| NEG | Changes the sign of the Source and stores it in a designated address. |

In the next chapter, an example of using these instruction for scaling analog inputs or outputs will be shown. In fact, several methods will be shown, but I'll start with the most basic – which involves the direct use of math instructions – the MUL, DIV, and ADD or SUB. You may already be familiar with easier methods of scaling using of the SCL and SCP instructions, and these will be reviewed as well.

## Using the Math Instructions & F8 Data File Values:

- When using these instructions, the "Source" can be either a word address that contains a number, such as N7:10 from the N7 data file, or a constant that you program directly into the instruction.

- For SLC 5/03, /04 and /05 processors, the F8 “floating point” data file can be used if you are working with numbers that contain decimal place values.

- The F8 data file uses 2-word elements to represent a numeric value – remember that when using an F8 element you will be addressing a full 32 bits. Here again is the addressing syntax chart for all the data files.

- The F8, or any other floating-point data files that you add to your project, can have from 0 to 255 word elements (32 bit each). A typical address would be something like F8:2 or F12:5, where F8 or F12 designates the particular “floating point” data file, and the 2 or 5 reference the specific two word element.

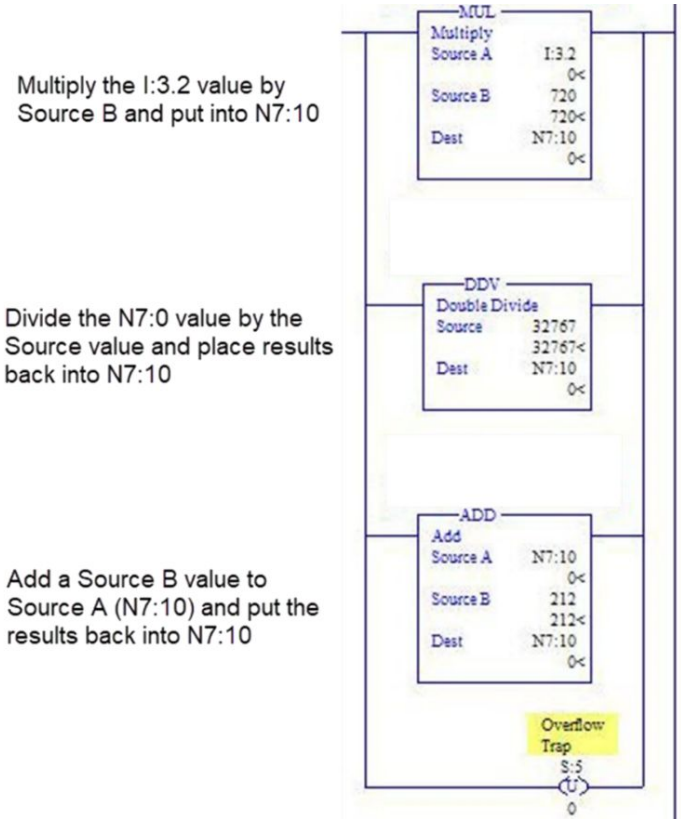
## Status File Bits:

When working with math instructions, it’s important to become aware of certain status file bits that directly monitor math registers. Doing so can help you avoid processor faults and downtime. Here are the main “status” bits that are linked to the math functions.

| Status File Bits | Descriptions |
|---|---|
| S:0/0 | This is the "Carry" or "C" bit. It is set if a carry over (ADD instruction) or a borrow (SUB instruction) is generated during execution of the instruction. |
| S:0/1 | This is the "Overflow" (V) bit. It is set if either an "overflow" or "underflow" is detected. |
| S:0/2 | This is the "Zero" (Z) bit which sets if the result is zero, otherwise it is reset. |
| S:0/3 | This is the "Sign" (S) bit. It sets if the result is negative. |
| S:5/0 | This is the "Overflow Trap" bit. It is set when a math overflow is detected or division by zero. Will cause processor fault and must be reset. |
| S:2/14 | This is the "Math Overflow Selection" bit. You can set this bit to "1" if you need to use 32 bit addition and subtraction. |
| S:13 | For the MUL instruction S:13 contains the "least" significant word of a 32 bit value. For DIV and DDV it will contain the "remainder". |
| S:14 | For the MUL instruction S:14 contains the "most" significant word of a 32 bit value. For DIV and DDV it will contain the quotient. |

Here is a short example of some math instructions used in an actual program. As they are executed, they create a fault which sets Status Bit S:5/0, the Overflow Trap bit.

After all three math instructions have acted-upon the N7:10 integer value, the large number created by the MUL instruction, which created a fault condition, no longer exists. At this point the S:5/0 bit can simply be unlatched. If unlatched before the end of the scan – no processor fault occurs.




Synopsis on Execution on this Scaling Logic:

- The analog input value, coming into I: 3.2 input, is “translated” from a 0-10 vdc signal into an integer value which ranges from 0 to 32767. This 0 to 32767 is called the “raw input data” and is what you would see if you look directly into the input data file for the I:3 slot on channel 2. It is also the numeric value that the processor uses for whatever actions are called for by program instructions.

- So the first instruction in this branched rung, the MUL, takes the raw data from I:3.2 – currently at 0 and multiplies it by “source B”, a constant of 720. It then places the result in the integer destination N7:10. If the I:3.2 input was at 10 vdc, the raw data translated by the input card could be 32767. When multiplied by our constant of 720 the generated number would far exceed the maximum for a 16 bit number, and the S:5/0 bit is set.

- The next instruction, a DDV, takes the N7:10 integer value, now residing in the 32 bit “math register” of the Status data file, and divides it by the constant of 32767. It places the resulting (quotient) back into the N7:10 address.

- The ADD instruction then adds “source B”, which is the value in N7:10, to a constant of 212, and once again places the answer back into the N7:10 address.

- The overflow trap bit will be set to “1” any time a math instruction is executed that results in a number larger than 32757 or a number being divided by 0.

## Advanced Math Instructions:

In the previous chart I've included some of the most commonly used math instructions – but there are many more you may find useful. Here is a chart of some other instructions that you can use for more advanced calculations.

| Instruction | Descriptions |
|---|---|
| SWP | Swaps the low and high bytes of a specified number of words of a bit, integer, ASCII, or string file. |
| ABS | Computes the absolute value of the source and places the result in designated destination address. |
| LOG | Computes the log base 10 of the value in the "source" and stores the result in a designated destination address. |
| LN | Computes the natural log of the value in the "source" and stores the result in a designated destination address. |
| SIN | Computes the sine of a number and stores the result in a designated destination address. |
| COS | Computes the cosine of a number and stores the result in a designated destination address. |
| TAN | Computes the tangent of a number and stores the result in a designated destination address. |
| ASN | Computes the arc-sine of a number and stores the result in a designated destination address. |
| ACS | Computes the arc-cosine of a number and stores the result in a designated destination address. |
| ATN | Computes the arc-tangent of a number and stores the result in a designated destination address. |

## Analog Scaling Instructions

The use of analog devices is so common in equipment and process control systems, that I felt an entire chapter devoted to this topic would be beneficial. There are specific instructions that will accomplish scaling, but at times it will be necessary to work out some math details in order to actually program an analog application. This was how the constant of 720 was derived in the previous example of scaling using math instructions. I also think that going through these calculations, will help you better understand the concept of what this type of routine accomplishes within a program.

An analog input is a “changing” voltage or current signal, sent to the PLC input module from some type of field device. This signal varies according to the physical conditions that act upon it, such as temperature, pressure, level of liquid in a tank, position of a work piece, RPM or any number of things that can be measured. The corresponding signal can be 0 to 10 Vdc, 0 to 5 Vdc, -10 to +10 Vdc, or a current signal such as 4 to 20 mA. These types of variable signals must interface with an analog module within the PLC rack, which then processes the input signal, and translates it into a value the processor can understand and use. Just as with other instructions covered thus far, the translation activity of an analog input channel places a numeric value into a 16 bit register or “word”- in binary format. The processor then uses this value during program execution.

The issue is that, if we happen to be using a 0 to 10 Vdc input, the resulting 16 bit number is, by itself, not very useful until it can be translated, used and shown in some meaningful way. For instance, if we are receiving 5 Vdc from

a pressure transducer out in the field, and our analog input card turns this voltage into a 16,384 integer value for the processor, we still need to establish, by programing, what that numeric value should do. For example; we could use it to cause a pump motor to run at full or half-speed, show some type of reading on an HMI panel, change a valve position, or output an alarm. The creation of “meaningful” relationships between the raw data coming into the processor - and how the program uses this data - is really the focus of scaling. Another thing that “scaling” is all about, is establishing a “linear” relationship between input data and output data – this chapter will cover this in detail.

Here are some basic definitions and terms commonly used when dealing with analog scaling and the programming issues that are involved.

## Definitions & Terms:

Analog Quantity: Physical quantity that varies continually between high and low limits.

Transducer: A field device that takes a physical analog quantity and changes it to an analog signal.

Analog Signal: Common analog signals produced by or sent out to field devices. Examples: 0 to 10 Vdc, -10 to +10 Vdc, 0 to 5 Vdc, 1 to 5 Vdc, 4 to 20 mA, -20 to +20 mA, and 0 to 20 mA.

Input Signal: The analog signal input to the module, for the 1746-NI4, on one of four channels. Each of these channels is addressed on a “word” level, for example: I:3.2 which is “slot 3”, “word element 2”.

Raw Data: The digital output from the module in binary format representing an integer value to the processor.

Scaled Output: A converted value that will be used for display or other purposes.

Note: Scaling is accomplished within the processor – not the input module. Scaling depends on our programmed instructions.

Analog Input Module: Converts the input signal into a digital representation that the processor can recognize and use. Serves as an A/D converter with output range dependent on how the module is configured. See example below.

Analog Output Module: Converts the digital data value from the processor and converts this binary number into a proportional output signal. Serves as a D/A converter for the specific type of output range selected and configured for the module.

Example: 1746-NI4 Module

Voltage / Current Range Integer Representation

-10 vdc to +10 vdc -32,768 to +32,767

0 to 10 vdc 0 to 32,767

0 to 5 vdc 0 to 16,384

1 to 5 vdc 3,277 to 16,384

-20 to +20 ma -16,384 to +16,384

0 to 20 ma 0 to 16,384

4 to 20 ma 3,277 to 16,384

With these terms in mind, let's take a good look at what scaling instructions are all about and how they are used to establish the relationship you desire between an input signal and its corresponding output. Usually it's desirable for this relationship to follow a linear pattern or equation, and it is often helpful to begin by drawing this out on paper.

## Example Application:

1746-NI4 Module, Channel 3, Address I: 3.2

Temperature Transducer: 0 to 10 vdc

Proportional Range (linear): 212 deg F to 932 deg F, (100C to 500C)

Process Temperature must stay within 500 deg F to 575 deg F.

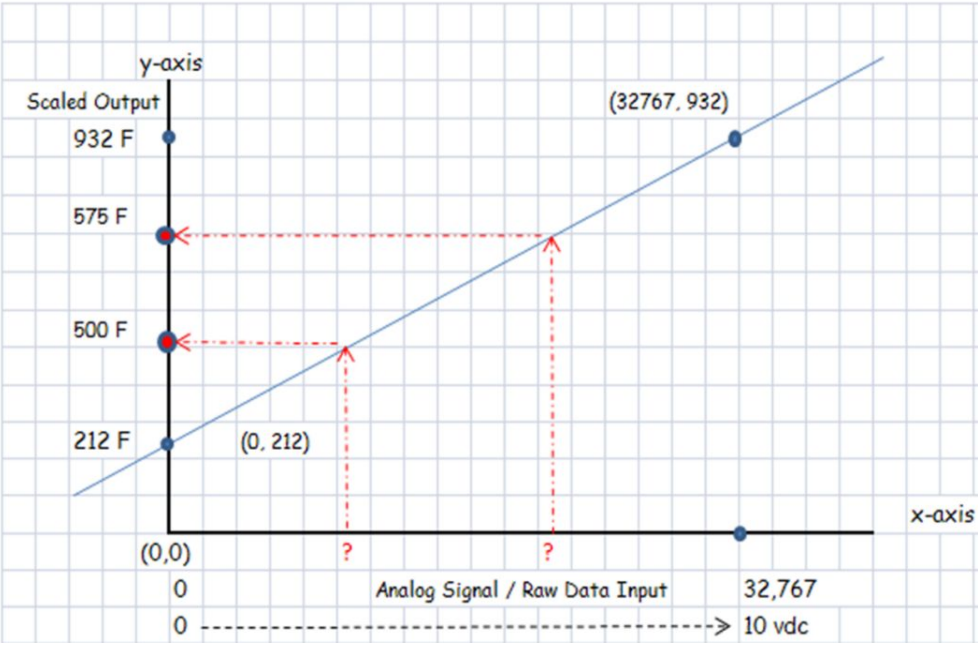
Objective: Set a High Alarm or Low Alarm if out of range.

Let's begin by drawing a sketch of the required linear function based on these details. Here is what we know:

- The transducer proportional range: 212 deg F to 575 deg F.
- Our desired control range: 500 deg F to 575 deg F.

- The 0 to 10 Vdc input to the module will correspond to 0 to 32767 values to the processor. Remember the module is acting as an A/D converter.

We want to know the raw integer values that equate to the control range so we can use these values to program the High and Low Temp alarms.




Along the bottom of our sketch, on the “x” axis, we can label the range of input voltage from the transducer, and also the “raw input” range the analog input module provides for the processor. This has been selected, by switch settings on the module, so we know that since 0 to 10 vdc is coming into our input card, this will correspond to a 0 to 32,767 range of values to the PLC processor.

We know what we want to accomplish on the output side of our graph, the “y” axis. We want to maintain a process within the 500 to 575 degree F temperature range, and set off alarms if the temperature goes higher or falls below this

range. Since we need to program for the alarms, and control the temperature within a range, the questions must be asked;

- “At what raw input values do we program the heat to cycle on and off?”
- “At what input values do we set off a Low Temp alarm?”
- “At what input values do we set off a High Temp alarm?”

Remember that our transducer, measuring the temperature value, provides a proportional range from temperatures of 212o F to 932o F. Any straight line or linear equation, such as the one sketched into the graph above, follows this equation, where “y” is the “scaled” output data corresponding to “y” axis levels.

$$y = mx + b$$

From this we can derive other equations to solve for the remaining values of the basic linear equation:

Where “x” is equal to our “raw data” from the analog input module.

$$x = \frac{(y - b)}{m}$$

Where “b” is the offset of where the line crosses the “y” axis above or below the “x” axis. This offset is often referred to as the “y-intercept”.

$$b = y - mx$$

Where “m” is the “slope” value, sometimes referred to as rise/run. It can be calculated by using our known inputs and outputs, the (x, y) coordinates, as follows:

$$m = \frac{(y_2 - y_1)}{(x_2 - x_1)}$$

While there are other methods of programming for the raw input values that will equate with a low of 500 and a high of 575 deg F, two of these methods still require some calculations using these derived equations.

First, we can calculate the “slope” or “m” value of our equation using our two known points: the (x, y) coordinates for our first point is (0, 212), and the coordinates for the second point – at the far end of our range, is (32767, 932).

These two points span, not just our desired temperature range of 500 to 575 degrees, but all of the values in the proportional range of our temperature transducer. So using the equation for “m” we can calculate the following:

$$m = \frac{(932 - 212)}{(32767 - 0)} = \frac{(720)}{(32767)} = 0.02197$$

Using the math instructions, we will use the 720 and the 32767 values, and when using the SCL instruction the decimal form for “m” will be used.

Next we can calculate the “b” offset value, even though it is pretty obvious on our graph, by plugging in the “m” value and also the (x, y) values for one of the points – either one

will give the same answer now that we know the value for slope “m”.

Using the point (32767, 932) for the (x, y) values, and the calculated slope (m), we verify the “b” value. Referred to as “offset” or “y intercept” value.

$$b = 932 - (0.02197 \times 32767)$$
$$b = 212$$

At this point, we have enough to calculate the “raw input” values for our temperature range. After doing this, we also have the data necessary to program control logic for the high and low temperature alarms, and heating control.

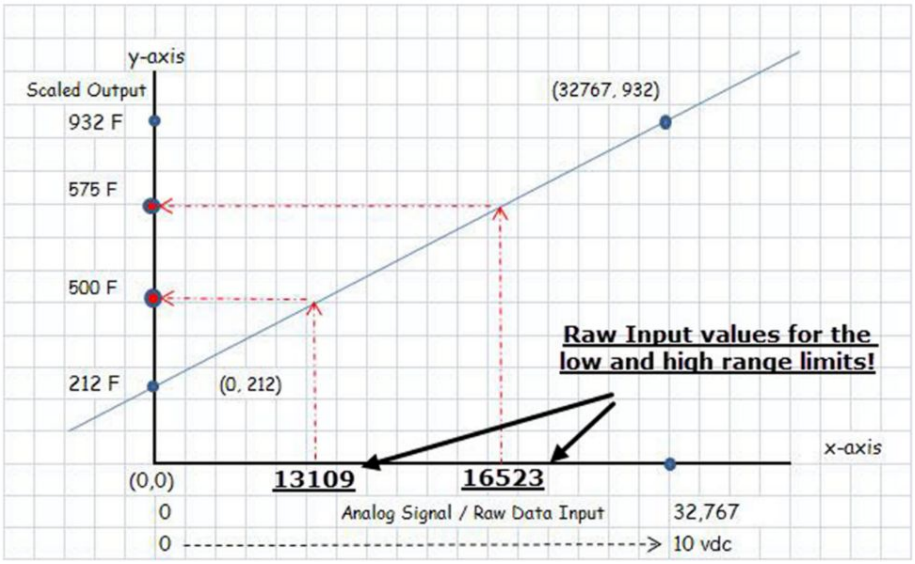
Using the derived equation to solve for “x”; we now can use our desired output “y” values , the offset value for “b”, and slope “m”, to calculate the following raw data values for the high and low limit alarms:

$$x = \frac{(y - b)}{m}$$

$$x = \frac{575 - 212}{0.02197}$$ Therefore: High Limit (raw input) = 16523

$$x = \frac{500 - 212}{0.02197}$$ Therefore: Low Limit (raw input) = 13109

Here is our complete graph showing scaled output values with the corresponding “raw input” integer values.



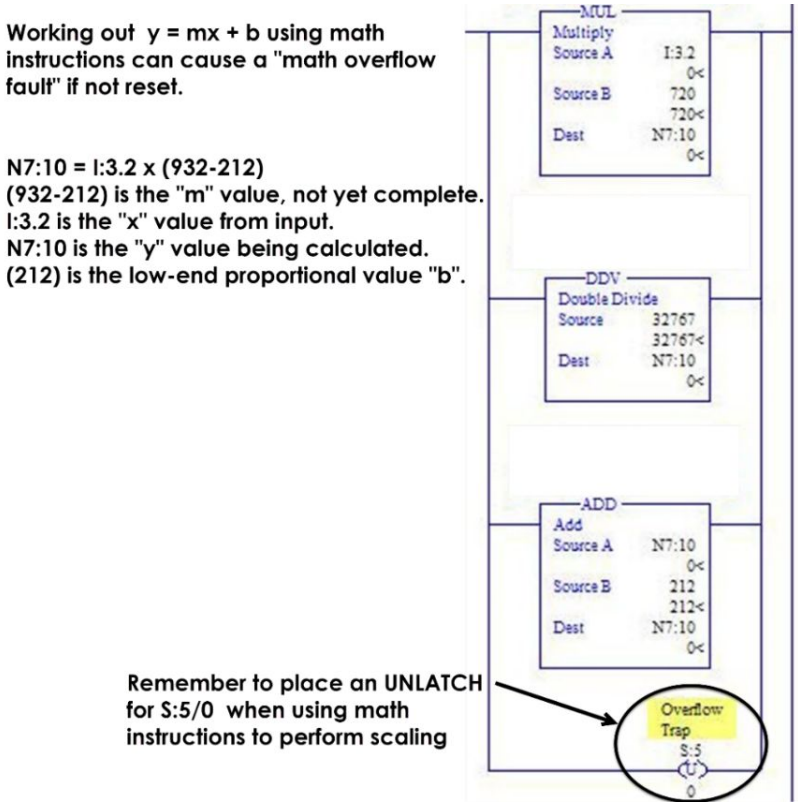

Using the “math” instructions discussed earlier, the actual programming might look something like this. This example of the math method of scaling, uses the MUL and DDV instruction. Remember that the DDV uses the 32 bits of the math register and provides a good method for the SLC family of processors.

Also remember, that when individual math instructions are used, it is a step-by-step process of programming as you “work-out” the “y = mx + b” linear equation, and it is still necessary to have already calculated values for slope and offset. As you can see in the program example; I used the fraction form of slope “m”, (720 / 32767), as shown on page 43, but performed the operation in two math steps. The MUL integrated the raw input value from the module, and then the DDV instruction completed the calculation for “m”. Thus; the first two instructions give us the value for “mx”.

$$y = mx + b$$

$$(N7:10) \text{ or } "Y" = \frac{932 - 212}{32767} (I:3/2) + 212$$

Note also the use of the S2:5/0 “overflow” unlatch discussed earlier – it must be reset to zero before the end of scan or a processor fault will occur.




In using the DDV or “double divide” instruction, remember that it utilizes words S:13 and S:14 - a total of 32 bits, referred to as the “math register”. This provides a temporary holding place for math calculations and obviously can hold a very large number. The “source” of the DDV

instruction always acts as the “divisor” for whatever is in the math register and stores the resulting quotient into the address you designate in the destination field.

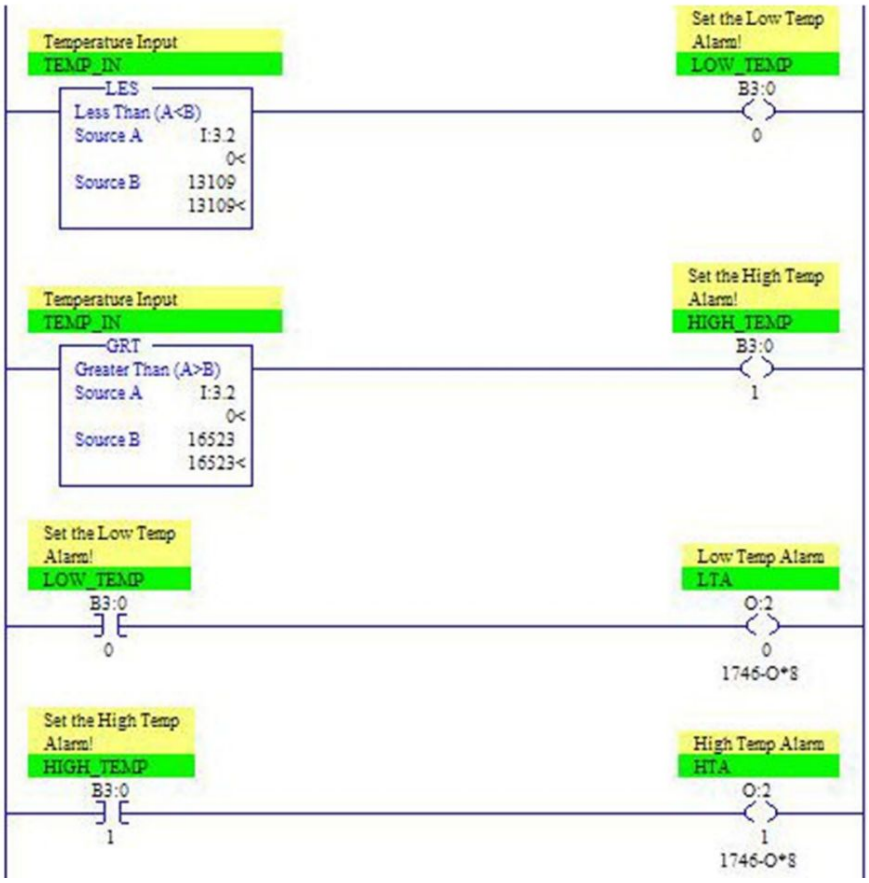
So the basic order of operations for these instructions:

- Using the “m” slope value in its “fractional” form - we wind up doing the multiplication first – the (720 x input value) – generating a number which is put into the math register.

- Then we follow by using the DDV which completes the “mx” part of the equation.

- All that is necessary at this point is to account for whatever offset value we have calculated by using the “ADD”, or in some cases a “SUB” instruction.

- Continuing with additional logic, we could use the I:3.2 input values – now that we know what they are, to program our high and low temperature range limits. And the N7:10 integer value, can be used for display purposes or for some other type of analog output control element.



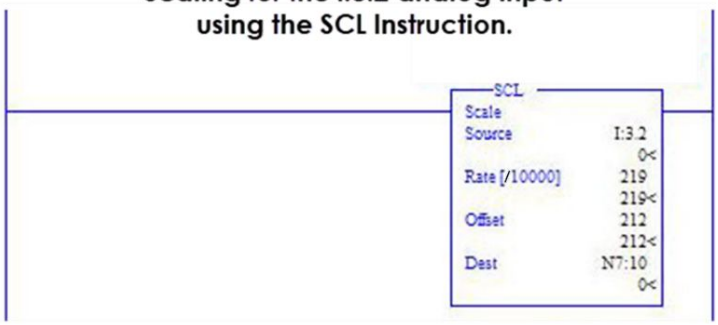

Using math instructions to accomplish scaled output values can be done with any of the SLC models. For “fixed” SLC 500 and 5/01 models, you must use the math method. If using a 5/02 or later model, you also have other options such as the use of using the “SCL” instruction, and the “SCP” instruction for 5/03, /04, and /05’s.

## The SCL – “Scaling” instruction:

Using the same scenario and calculations from our example, here is the programming for using the SCL instruction. It essentially takes the place of the three math instructions and the need to reset status bit S2:5/0. We have our scaled

value in the N7:10 address and the remaining rungs of programming are the same.

Scaling for the I:3.2 analog input using the SCL Instruction.



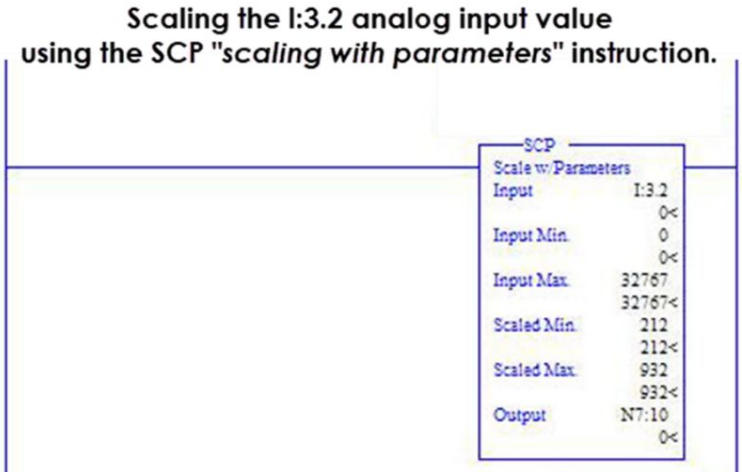

Note that since the slope value is going to be divided by 10000 within the instruction, you must multiply the “m” value (0.02197) by 10000, and enter this value into the “rate” field. The value of the “b” offset (212) is entered as well.

## The SCP – “Scale with Parameters” instruction:

One of the easiest methods of “scaling” is to use the SCP instruction; however it is only available for SLC models 5/03, 5/04 and 5/05 and operating system (OS401). This instruction does the slope and offset calculations for us, and provides for a linear relationship between our input range and our scaled output values.

Even though this is the case, I still believe it’s helpful to sketch or graph the application, and do the math to get a better visual of exactly what you want to accomplish with your ladder logic. This can also be helpful when troubleshooting issues on existing PLC controlled equipment.

Here is an example of this instruction used with previous application.




We designate the input address for the incoming signal, I:3.2, plug in the values for the full range of raw input data – minimum 0 to a maximum of 32767. We then plug in values for the scaled output – our full proportional or “linear” range – 212 as the minimum and 932 as the maximum. Finally, enter the address for the scaled output value, in our example it is addressed to N7:10.

Note that, just as in all three examples, we end up having a “scaled” output value in the N7:10 integer data file. Once again, we can use this value in many different ways, for display on a graphics panel or HMI, to control the speed of a variable frequency drive motor, or to control a heating system, are just a few possibilities.

## Analog Output Modules:

These modules convert the digital (binary) data value from the processor into a proportional output signal; and thus

serves as a D/A converter for the specific output range selected and configured for the module.

These modules do basically the same thing as an analog input - only in reverse. They provide essentially the same voltage and current levels from given integer ranges (received from the processor) as do the input modules.

Example: 1746-NO4V, 1746-NO4I Modules

Voltage / Current Range Integer Representation

(-10) to (+10) vdc -32,768 to +32,767

0 to 10 vdc 0 to 32,764

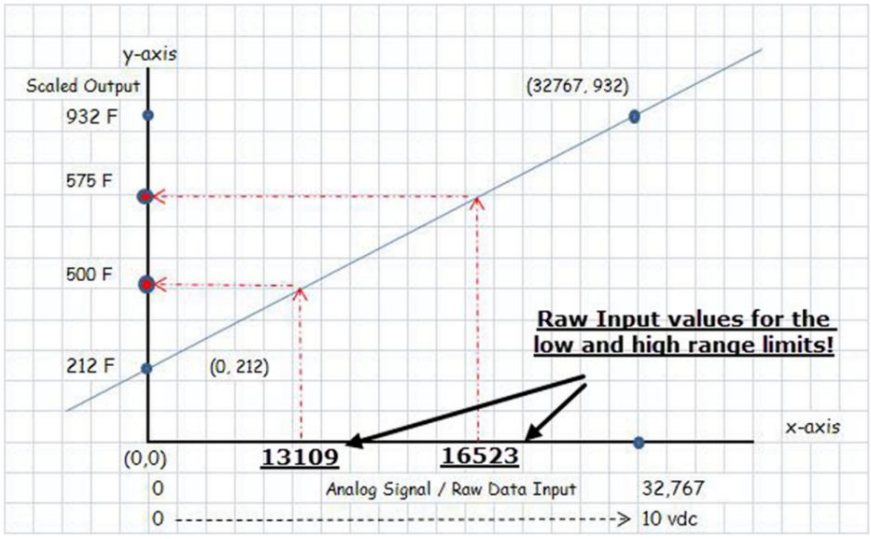
0 to 5 vdc 0 to 16,384

1 to 5 vdc 3,277 to 16,384

0 to 20 ma 0 to 31,208

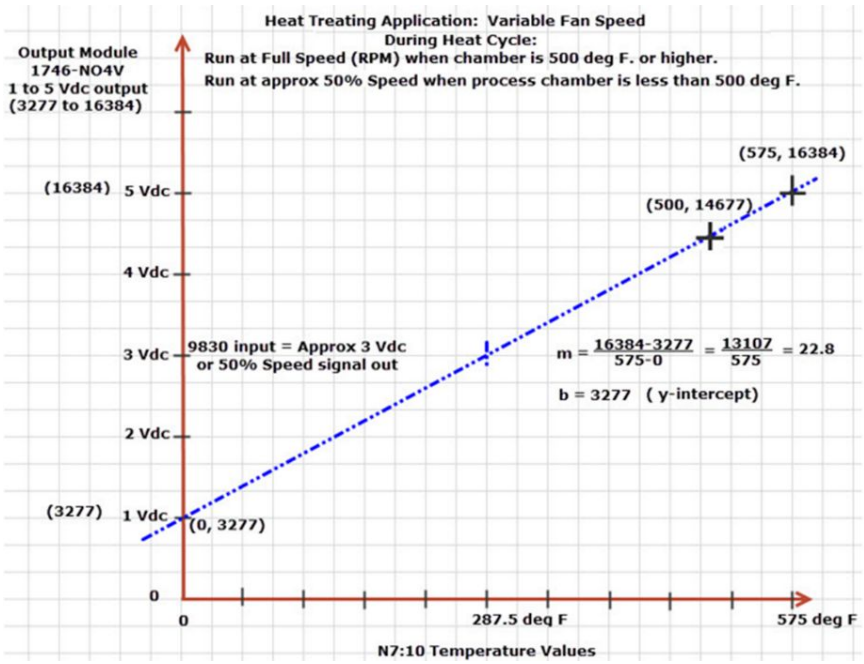
4 to 20 ma 6242 to 16,384

Here is the sketch from our previous example, showing the linear relationship for the controlled temperature range. It can be used in another scaling routine to program a 1 to 5 Vdc output signal that controls the variable speed of a circulation fan. This fan speed is dependent upon temperature inside a process chamber.




Here is a different diagram for an analog output module which we can use for the variable speed control of a circulation fan in our example heat treating process. Note that for the output module used, the 1746-NO4V, for a 1 to 5 Vdc output from the module, the integer range is 3,277 to 16,384. These integer values are the input values sent to the module from the processor, and the output voltage from the module will vary from 1 to 5 Vdc.

By using the temperature values within the desired control range along the “x” coordinate axis, and these integer values of the module along the “y” axis, we come up with a diagram that looks something like this.



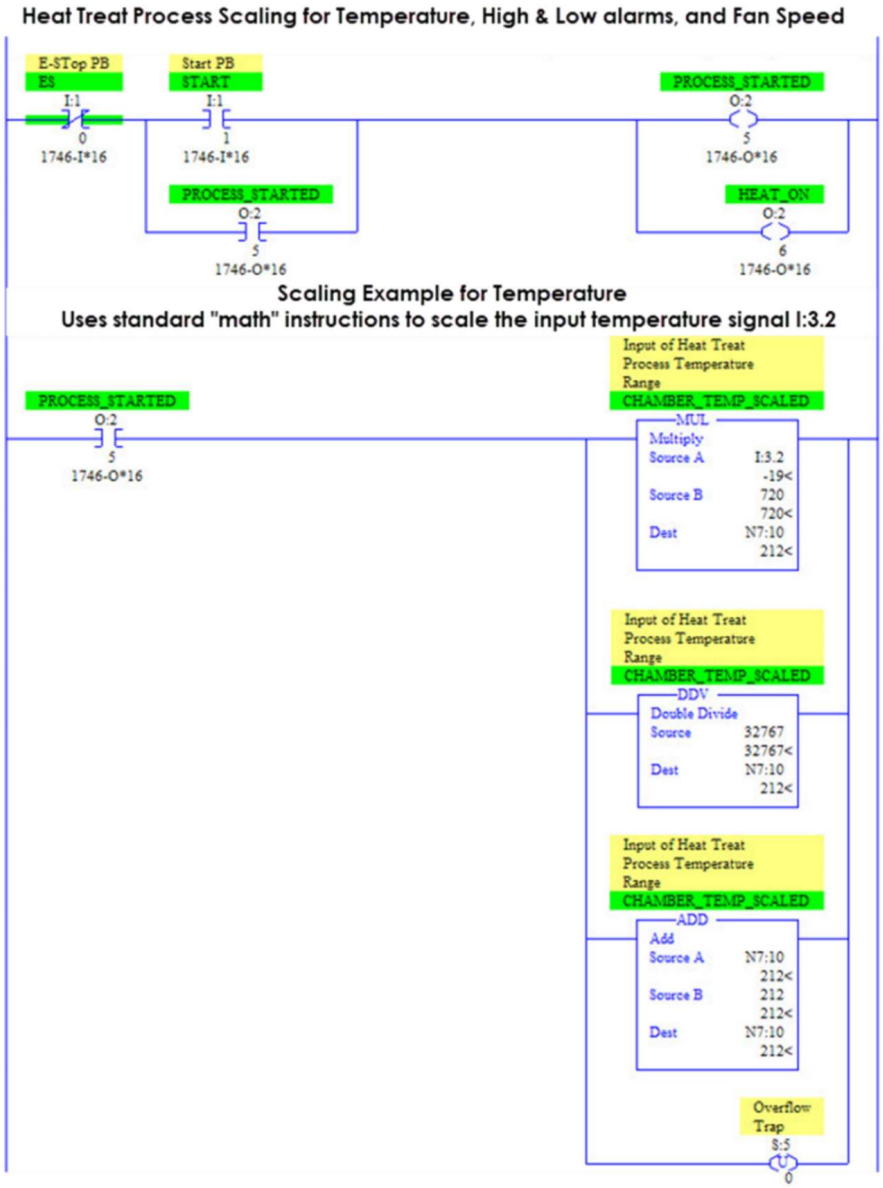

Once again, the linear equation “y = mx + b” is calculated using the (x,y) coordinates of the full range of the module settings. Values are obtained for the slope “m”, and also for the “b” (y-intercept) or offset value.

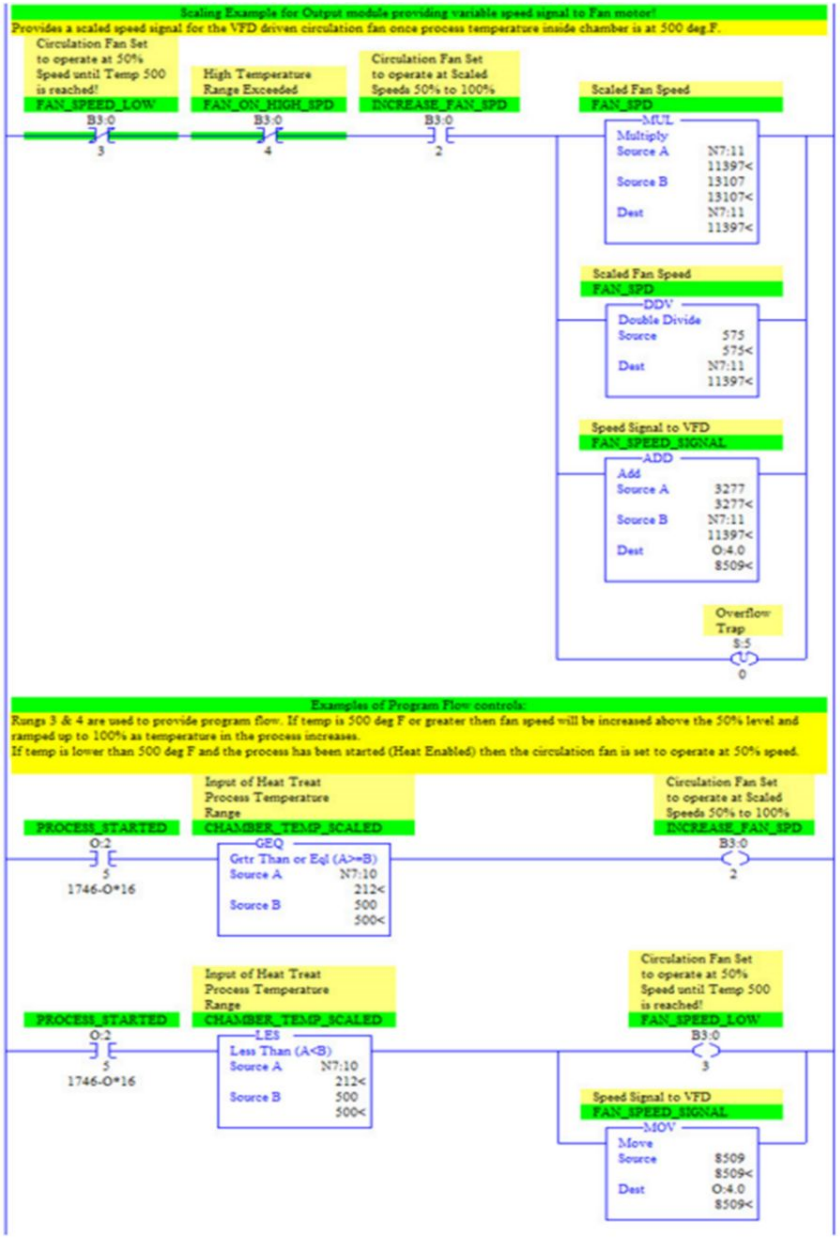
As always, there are many different ways and methods to accomplish objectives when programming. However, it’s not an easy task to figure out what someone did if they have failed to document with descriptions and symbols. Trying to figure out what someone did with an analog scaling application can be particularly challenging, so good comments and documentation is an important issue.

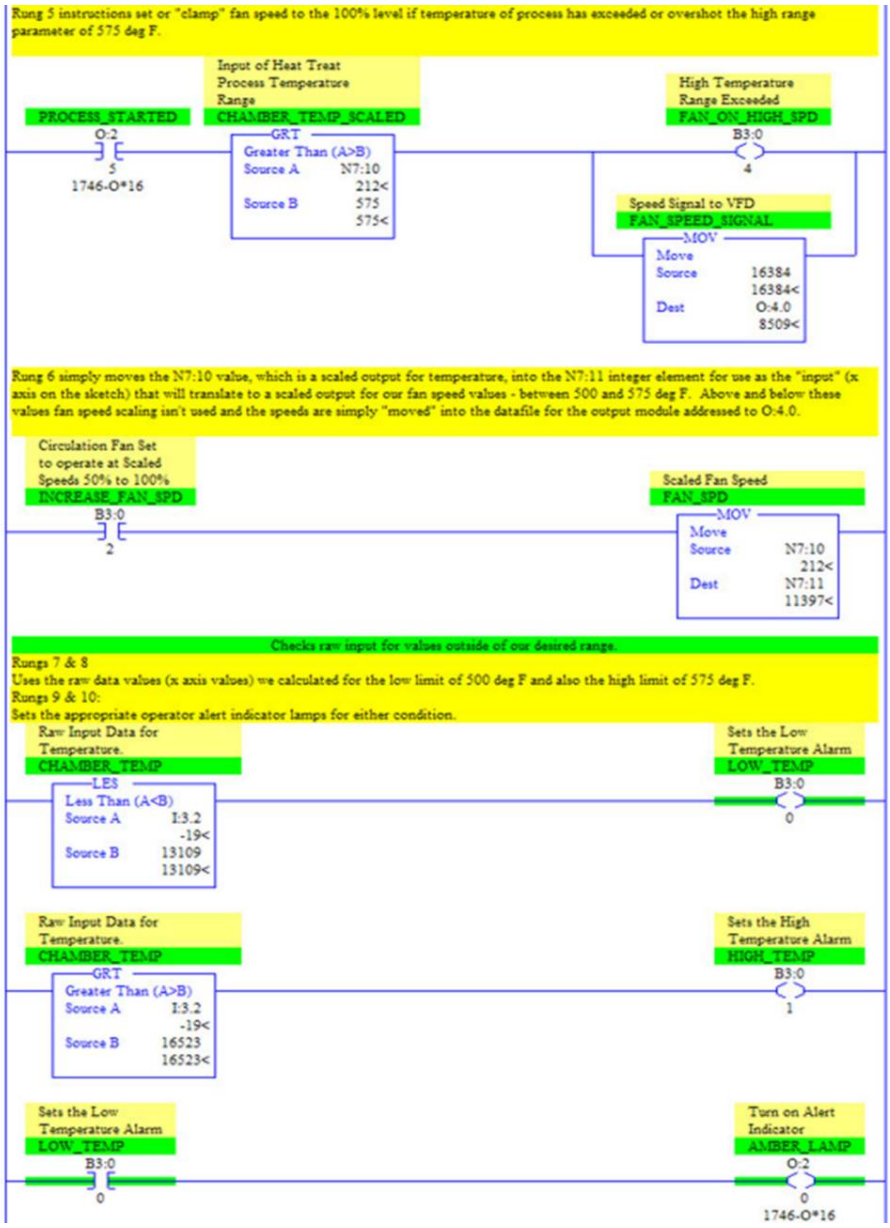
I’ve tried to follow my own advice in documenting the following example program, perhaps overdoing it a bit. But as you can see, it’s helpful in understanding what can be done in programming a control system. You will likely see many other methods of accomplishing this type of control

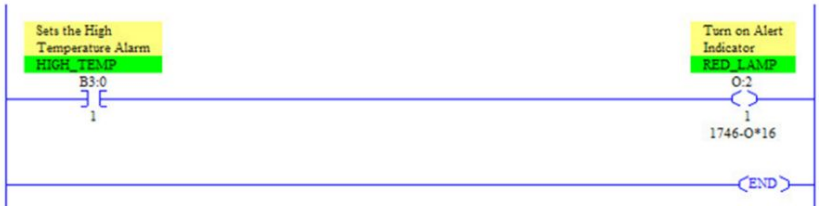
scenario, but for now my focus is to give some basic examples of scaling for analog modules, both input and output types.

Here is a simple 11 rung example of programming the above application:

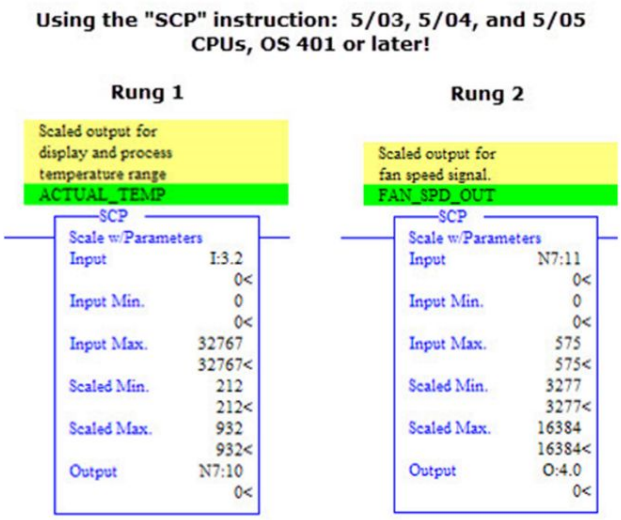



Scaling Example for Output module providing variable speed signal to Fan motor!
Provides a scaled speed signal for the VFD driven circulation fan once process temperature inside chamber is at 500 deg.F.
Circulation Fan Set to operate at 50% Speed until Temp 500 is reached!
FAN_SPEED_LOW
B3:0
3
High Temperature Range Exceeded
FAN_ON_HIGH_SPD
B3:0
4
Circulation Fan Set to operate at Scaled Speeds 50% to 100%
INCREASE_FAN_SPD
B3:0
2
Scaled Fan Speed
FAN_SPD
MUL
Multiply
Source A N7:11 11397<
Source B 13107 13107<
Dest N7:11 11397<
Scaled Fan Speed
FAN_SPD
DDV
Double Divide
Source 575 575<
Dest N7:11 11397<
Speed Signal to VFD
FAN_SPEED_SIGNAL
ADD
Add
Source A 3277 3277<
Source B N7:11 11397<
Dest O:4.0 8509<
Overflow Trap
S:5
0
Examples of Program Flow controls:
Rungs 3 & 4 are used to provide program flow. If temp is 500 deg F or greater then fan speed will be increased above the 50% level and ramped up to 100% as temperature in the process increases.
If temp is lower than 500 deg F and the process has been started (Heat Enabled) then the circulation fan is set to operate at 50% speed.
PROCESS_STARTED
O:2
5
1746-O*16
Input of Heat Treat Process Temperature Range
CHAMBER_TEMP_SCALED
GEQ
Grtr Than or Eql (A>=B)
Source A N7:10 212<
Source B 500 500<
Circulation Fan Set to operate at Scaled Speeds 50% to 100%
INCREASE_FAN_SPD
B3:0
2
PROCESS_STARTED
O:2
5
1746-O*16
Input of Heat Treat Process Temperature Range
CHAMBER_TEMP_SCALED
LES
Less Than (A<B)
Source A N7:10 212<
Source B 500 500<
Circulation Fan Set to operate at 50% Speed until Temp 500 is reached!
FAN_SPEED_LOW
B3:0
3
Speed Signal to VFD
FAN_SPEED_SIGNAL
MOV
Move
Source 8509 8509<
Dest O:4.0 8509<


Rung 5 instructions set or "clamp" fan speed to the 100% level if temperature of process has exceeded or overshot the high range parameter of 575 deg F.
Input of Heat Treat Process Temperature Range
CHAMBER_TEMP_SCALED
PROCESS_STARTED
O:2
5
1746-O*16
GRT
Greater Than (A>B)
Source A N7:10 212<
Source B 575 575<
High Temperature Range Exceeded
FAN_ON_HIGH_SPD
B3:0
4
Speed Signal to VFD
FAN_SPEED_SIGNAL
MOV
Move
Source 16384 16384<
Dest O:4.0 8509<
Rung 6 simply moves the N7:10 value, which is a scaled output for temperature, into the N7:11 integer element for use as the "input" (x axis on the sketch) that will translate to a scaled output for our fan speed values - between 500 and 575 deg F. Above and below these values fan speed scaling isn't used and the speeds are simply "moved" into the datafile for the output module addressed to O:4.0.
Circulation Fan Set to operate at Scaled Speeds 50% to 100%
INCREASE_FAN_SPD
B3:0
2
Scaled Fan Speed
FAN_SPD
MOV
Move
Source N7:10 212<
Dest N7:11 11397<
Checks raw input for values outside of our desired range.
Rungs 7 & 8
Uses the raw data values (x axis values) we calculated for the low limit of 500 deg F and also the high limit of 575 deg F.
Rungs 9 & 10:
Sets the appropriate operator alert indicator lamps for either condition.
Raw Input Data for Temperature.
CHAMBER_TEMP
LES
Less Than (A<B)
Source A I:3.2 -19<
Source B 13109 13109<
Sets the Low Temperature Alarm
LOW_TEMP
B3:0
0
Raw Input Data for Temperature.
CHAMBER_TEMP
GRT
Greater Than (A>B)
Source A I:3.2 -19<
Source B 16523 16523<
Sets the High Temperature Alarm
HIGH_TEMP
B3:0
1
Sets the Low Temperature Alarm
LOW_TEMP
B3:0
0
Turn on Alert Indicator
AMBER_LAMP
O:2
0
1746-O*16


Sets the High Temperature Alarm
HIGH_TEMP
B3:0
1
Turn on Alert Indicator
RED_LAMP
O:2
1
1746-O*16
END

For models 5/03, /04 and /05's with the newer operating system, you also have the option, just as shown earlier in the chapter, of using the SCP or "scale with parameters" instruction. Here are the instructions as they would be if used in the above ladder logic, as you can see, this instruction simply takes the place of the individual math instructions.




## The "PID" Instruction:

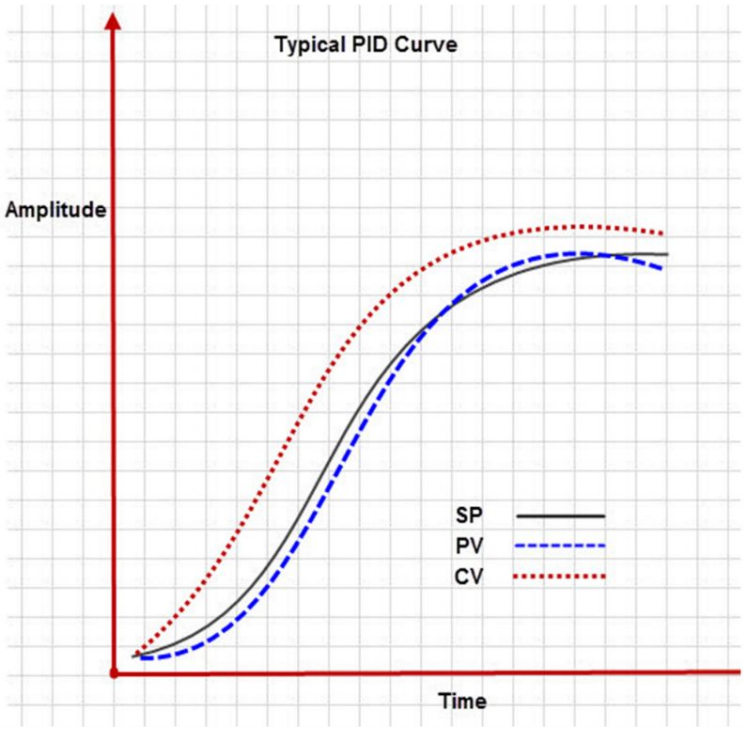
On a final note, another instruction often used in conjunction with analog control applications is the "PID" instruction. It is an output "action-performing" instruction composed of a 23 word block which holds parameters and information that are application specific and determined by the programmer.

PID control is a method of taking the difference between the process variable (PV) and the desired set point (SP), and bringing that difference, referred to as "error" - to essentially a zero difference.

This is accomplished by introducing a control variable (CV) into the process. With PID, the “P” stands for “proportional”, the “I” for “integral”, and the “D” for “derivative”, all calculus terminology and each one bringing unique characteristics of control into the situation.

The simple description of the PID instruction is that it receives an input value, the PV or process variable, checks it against the SP, and then provides an “output” or “CV” that will correct remaining “error” between the two. Because the input received and the resulting output of the instruction’s calculations are “variable” or constantly changing, this instruction is often used in conjunction with the other types of scaling instructions covered in this chapter. These will provide the input for the PID instruction, and also handle the resulting output, that leaves the PLC as some form of analog signal which will drive a control device.

The examples shown thus far, have followed scaling methods and instruction usage which are based on maintaining a “linear” or “straight-line” relationship between input signals and output data values. This however is not the case with PID control, as you can see from the following, the PV, and the CV that alters it, are anything but linear.




Most of the PID control systems that I've been involved throughout my career, have been related to the controlled application of heat, pressure and vacuum in heat treating and curing processes. These types of applications, and many others, require "tight" control of heat and the length of time in process; so rates of change, the amount of "overshoot", and even the cool down rates become very important. These are areas in which properly tuned PID loops excel.

On the other hand, regular scaling instructions, such as the SCL, SCP or "math" instructions, are the tools of choice when dealing with simple relationships where values are stable or set by the operator HMI and don't require constant oversight by a control algorithm such as the PID.

At this point, some of the main details to remember are:

- The PID normally controls a closed loop using input from an analog input module, the "process variable" or PV, and

provides an output “control variable” CV – sometimes referred to as the “manipulated variable”, to an analog output module.

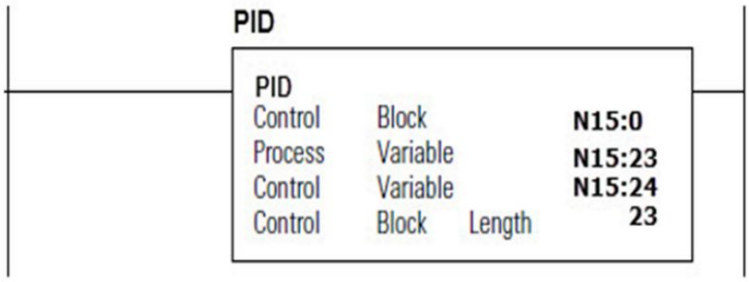
- Its calculations work to hold a PV at a desired “set point” or SP.

- By setting PID parameters, you can control items such as gain, whether the control is forward or reverse action, the “timing” used to update data tables, and many other important details related to “closed loop” PID control.

- These are the details stored within the 23 word control block for the PID instruction.

- Here is an example of the PID in a ladder logic program. Notice that it is “unconditionally” active as an output or “action performing” instruction, and that its addressing is done at the “word” level.

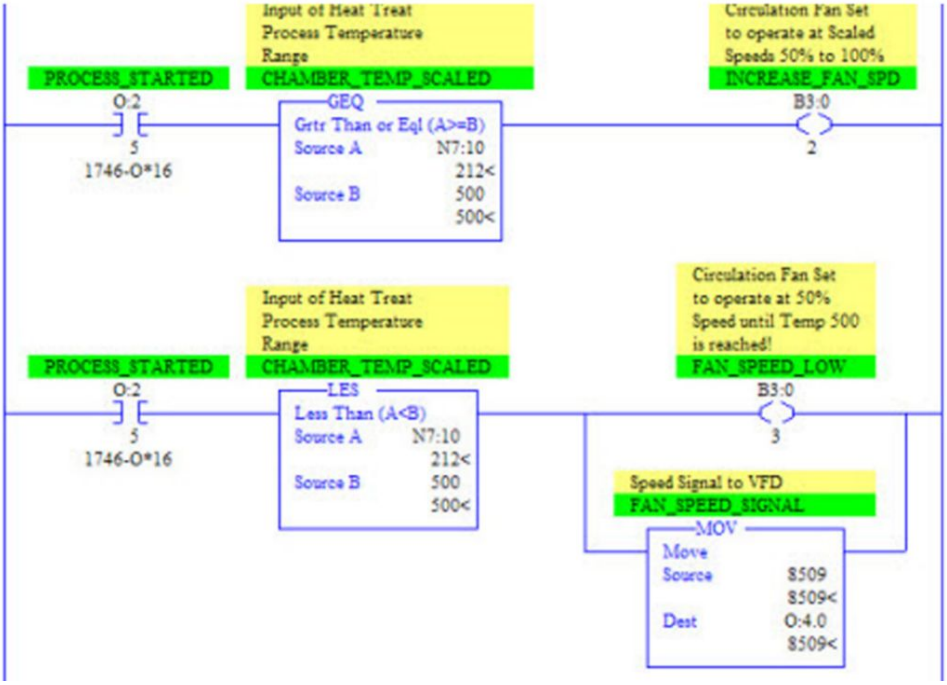
- The PV and CV will contain integer values that range from 0 to 16383, so this is how they are “scaled” for the incoming input (PV), and for the output module side, the (CV).



For greater detail on using this instruction and the specific programmable parameters go to the SLC 500 / RSLogix 500 Instruction Set: PUB 1747-RM001G-EN-P.

## Using Comparison Instructions:

In the previous program example you noticed several examples of “comparison” instructions. These “question – asking” conditions are “true” when their conditions are met, and pass logical continuity down to whatever “action-performing” output is at the end of the rung. Here are some examples from the previous chapter:

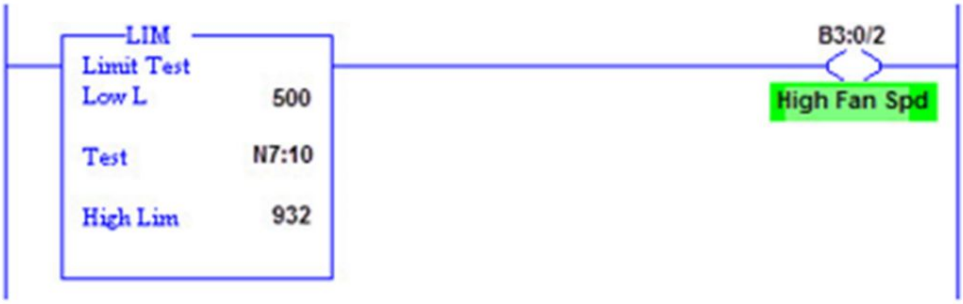



The comparison instructions can be used in a variety of interesting ways. In the previous example, I used them to qualify and set a bit for certain alarms and to establish fan speeds at predetermined levels of temperature. They are also often used to qualify operator selections, such as with a three position switch or BCD input selections, and can control the program flow to specific sections of ladder logic or subroutines.

In these example rungs, you see the GEQ – “greater than or equal to”, and the LES – “less than” instructions. Other comparison instructions that are part of the SLC-500 instruction set are the GRT – “greater than”, LEQ – “less than or equal to”, the EQU “equal to”, NEQ – “not equal”, the MEQ – “masked equal”, and the LIM – “limit test” instruction.

As you can see, these can be used to compare integer words, constants, or a single bit. The basic parameter constructs for the GRT, GEQ, LES, LEQ, EQU, and NEQ, are that the “source A” parameter must be an integer address, while “source B” can be either an address or a constant.

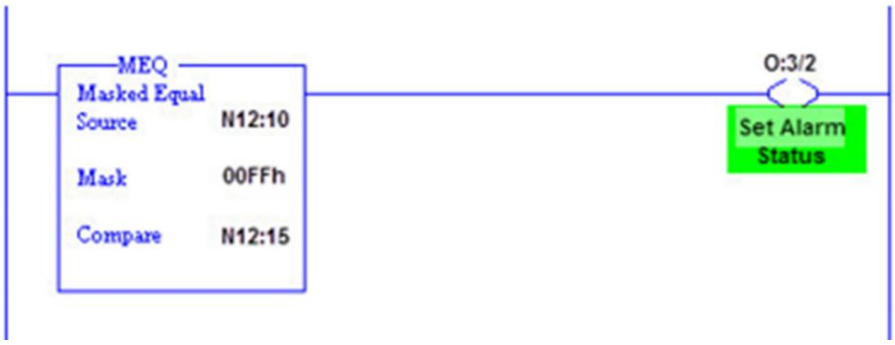
The LIM, or “limit test” does exactly as the term implies. If the test value is within the range of a high and low limit, the instruction becomes “true”. If the “test parameter” is entered as an address, then the high limit and low limit can be either an address, such as N7:11, or simply a constant. If the “test parameter” is entered as a constant, then the high and low limits must be entered as addresses. Here is the basic logic rung showing how this instruction might have been used in the programming example from the last chapter.



Where N7:10, a scaled value for temperature input, is used to send the speed signal to the VFD driven circulation fan. If scaled temperature is between the low and high limits of the proportional range, the instruction is “true” and the “high fan spd” bit is turned on.

Another usage for the LIM instruction is when the Low Limit is set to a value higher than the High Limit. In this scenario, when the Test value is between the High and Low values, the Limit instruction evaluates as “false” and doesn’t pass logical continuity. When the test value is equal to either the High or Low set-points, or outside of the Low – High range, it is evaluated as “true”.

The MEQ, or “masked equal”, is an instruction which allows comparison of two integer values while also allowing you to exclude certain bits from comparison. The basic instruction looks like this:




The “mask” value may be designated by an integer address, or as shown in this example, by a hexadecimal number, such as 00FF, which would be the equivalent binary number 0000 0000 1111 1111. The two values that you are comparing, the “Source” and “Compare” integers, filter through the “masking” value in a type of “AND- ing” operation. Bits which are to be excluded from the

comparison are set to “0” rather than a “1” in the mask value. Therefore, in the above example – if we had designated “00FFh” as the mask value, the first 8 bits of the “source” and “compare” values would be filtered through the mask and bits 8 – 15 would be ignored (masked) from the comparison. If the “filtered” results are equal, then the instruction is “true”. Here is an example of how this masking operation works.

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Source N7:10 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| Mask **00FFh** | **0** | **0** | **0** | **0** | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| Result of "AND-ing" | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| | | | | | | | | | | | | | | | | |
| Compare N7:15 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| Mask **00FFh** | **0** | **0** | **0** | **0** | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| Result of "AND-ing" | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| Results are equal so instruction is "true". | | | | | | | | | | | | | | | | |

Some of these comparison instructions, such as the GEQ or LEQ are easy and widely adaptable to different uses in your programs, while others, such as the MEQ may seldom be used. The point I want to emphasize, is that there are many instructions that can be applied to almost any conceivable programming scenario. Not all will be covered in this short book, so please take the time to become familiar with the SLC 500 instruction set for additional descriptions and examples detailing how these instructions can be implemented into your ladder logic programs.

## Data or Register Instructions:

In this chapter we'll look into a few of the "data handling" instructions. Here is a listing which includes several. The important detail to remember as you consider these instructions is to be aware that they act upon entire word elements of data - and in some cases multiple words.

| Instruction Mnemonic | Instruction Name | Description |
|---|---|---|
| COP | Copy File | Copies data from a source file to a designated destination file. |
| FLL | Fill File | Load a source value into each position of a designated destination file. |
| MOV | Move | Moves the source value to a designated destination address. |
| MVM | Masked Move | Moves data from a source location to a selected portion of a destination location |
| CLR | Clear | Technically listed with the "math" instructions, but handy for turning all the bits of a word to zero. |
| TOD | Convert to BCD | Converts a source integer value to BCD format and stores in designated destination address. |
| FRD | Convert from BCD | Converts a BCD source value into an integer value and stores in a designated destination address. |
| | | |
| | | |
| | | |

## The MOV Instruction:

In the previous scaling application, you were able to see the "MOV" instruction being used to move a value into an integer address, and also into an analog output address.

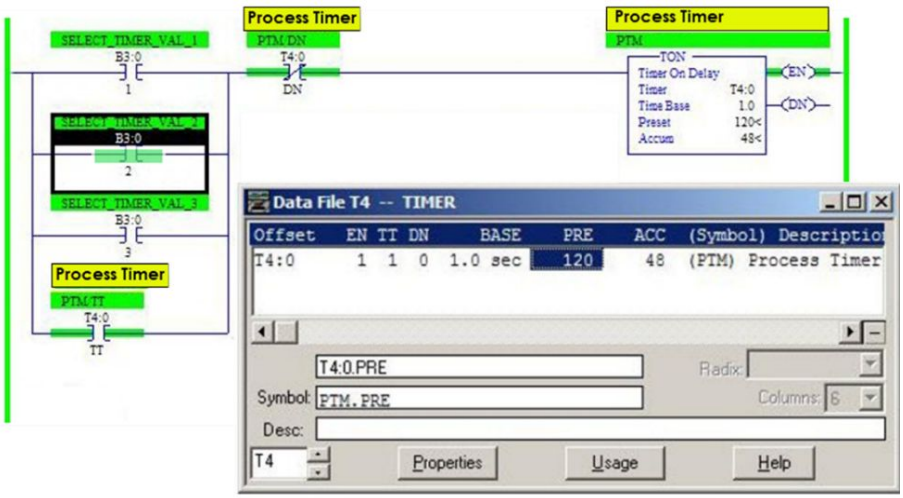
When using the MOV instruction, the designated “source”- whether an integer value or constant - is copied into a destination address. With 5/03 (OS301), 5/04 (OS401), and 5/05 processors, the source value can also be “floating-point” value or an ASCII string.

What makes the MOV such a versatile programming tool, is its ability to move an entire register of 16 bits at one time. Because of this functionality, it can be used to change the “Preset” values of timers and counters. You should recall that with these instructions, a word is designated for both the “preset” and “accumulator” values. The MOV can also be effectively used for input or output data-file registers to turn on all 16 outputs at once – basically with a single rung of ladder logic code.

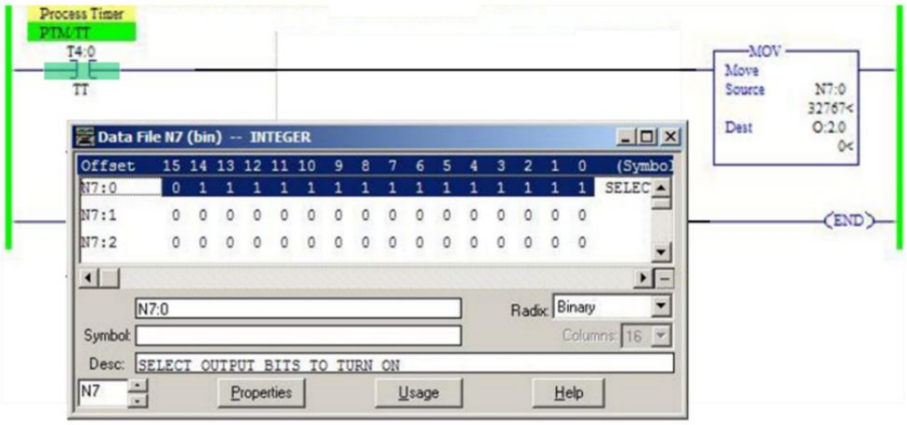
## The CLR Instruction:

In the following example I will show the use of an MOV to set timer “preset” values, and also use the “CLR” instruction to turn off a block of outputs. The B3 file bits 1, 2, 3 are used to simulate a momentary type of selector for the desired timer values of one, two, or three minutes. Once the T4:0 timer starts its timing cycle, the outputs of the slot 2 module turn “on” because the N7:0 integer value is set at 32767, which sets the 0 thru 14 bits to “1”. You could of course be very selective in your choice of exactly which bits / outputs to turn on by changing the value to less than 32767, but this is just an example of what can be done. When the timer is finished timing, it sets the T4:0/DN control bit which initiates the “CLR” instruction and turns these outputs “off” until another time value is selected.

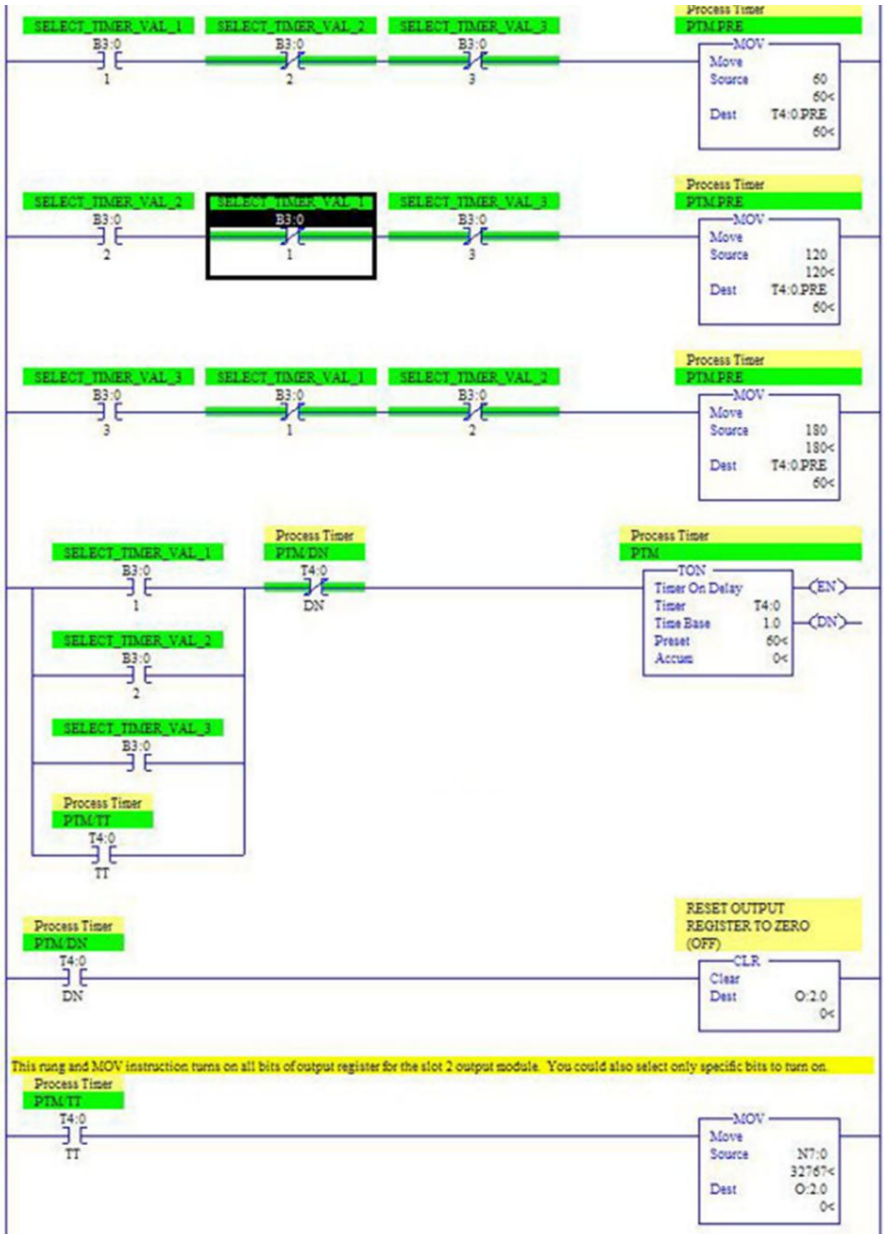
Here also are examples of the data files for the timer – set at the 2 minute selection, and the N7:0 file element shown as binary after the value of 32767 has been moved into the word.



**This Rung and MOV instruction turns-ON the bits of the Output Module in slot 2.**
**This method could also be used to turn on only select or specific bits and output points.**



Here is the example ladder logic. Remember that the B3 bits are simulating “momentary” rather than “maintained” inputs.



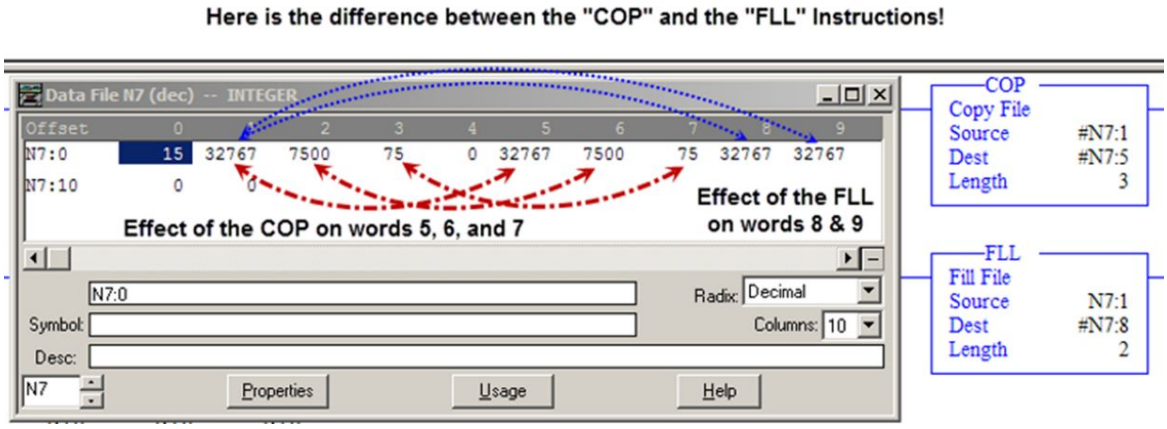
## The COP Instruction:

The “COP” acts much the same as the “MOV”, except that it moves sequential “blocks” of information rather than a single word. It copies the information into another sequential block of registers. The instruction “source” is the starting address of the file(s) you want to copy and the “destination” is the starting address of where the copy will be placed in storage. The “length” parameter designates the number of registers or “elements” that will be copied.

## The FLL Instruction:

Takes the value in the source address and copies it into the destination address, and also into sequential word elements if you have designated a “length” of more than one word.

Here is an example of the instruction programming and N7 data file after the rung is active.




As shown in this comparison of these instructions; the COP takes the values of word elements N7:1, N7:2, and N7:3 and copies in the same order to N7:5, N7:6, and N7:7. The FLL instruction, takes the value of N7:1 and places this value into N7:8 and N7:9.

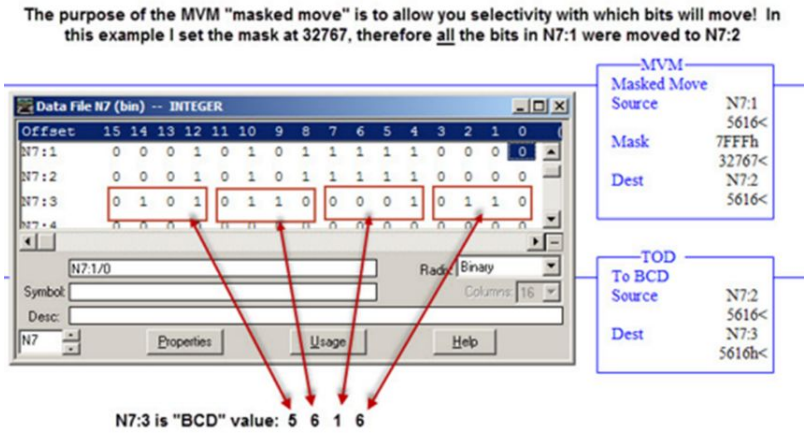
## The MVM, TOD, and FRD Instructions:

The MVM instruction, or “masked move”, works just like the “MOV” instruction shown earlier. The difference is that with the MVM it is possible to exclude certain bits from the move by filtering through a mask value. This performs what I would call an “AND-ing” operation, where if the value being moved has a bit set to “1”, AND the mask bit is set to “1” – then that bit value of “1” will be moved into the bit position of the destination word element. A zero in the bit position of the “mask” will not allow the move for the corresponding bit of the source value.

The TOD instruction converts an integer value into BCD or binary coded decimal format. This is a convenient method of programming for a seven-segment BCD display. Using the MOV instruction, a full word element or 16 bits can be moved to an output module for display of 4 digits. Remember that with BCD, each four bits of the word are equal to 8, 4, 2, 1 values – so each four bit group can represent and output a number from 0 to 9.

The FRD instruction, not shown in the following example, does the reverse operation as the TOD. It converts “BCD” source value into an integer value and places it into a designated address.

Here is an example of using these instructions and the corresponding N7 data file:




Note that when using the MVM, the mask can be a constant – shown here as a hexadecimal number “7FFFh” or 32767. It can also be an integer address in which you have set the value.

Also note the difference between the bits of N7:2 (binary) and N7:3 (BCD), even though both 16 bit registers represent the value of “5616”. The N7:3 bits could be moved directly into an output module to show the number 5616 on an operator panel display or HMI.

## Program Flow Instructions

Some of the most useful instructions in the SLC-500 instruction set are the program flow instructions. This is especially so when dealing with larger ladder programs, because these instructions allow you to write a program in blocks or sections that pertain to specific functions. One program I deal with from time to time has separate sections or “subroutines”, for the equipment’s “main control”, “safety resets”, “motor control”, “pressure control”, “temperature control” and “vacuum”. Building a program in subroutines makes it easier to follow and troubleshoot whenever that becomes necessary.

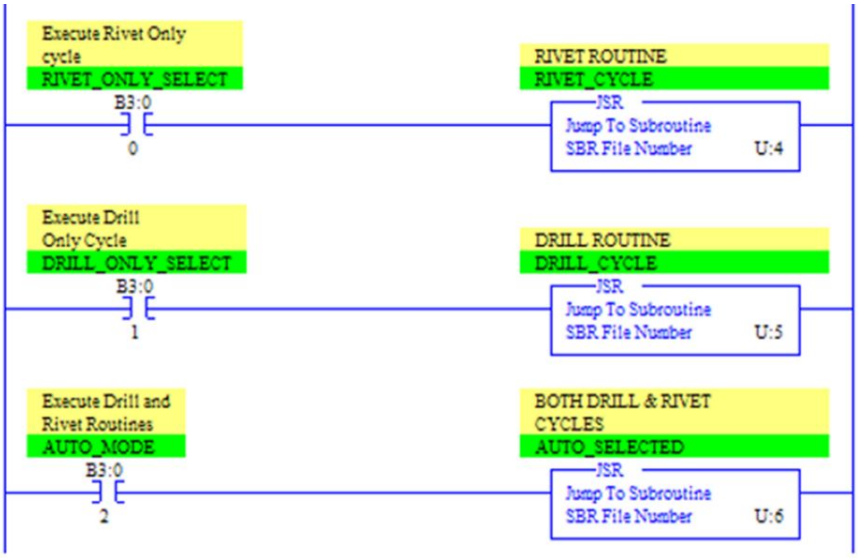
These instructions are used to control the sequence of execution of your program, allowing you to bypass the execution of certain rungs or to loop (repeat) within a certain block of programming until specific criteria is acceptable.

Another method of program control where these instructions become useful, is for equipment processes that can be used on different parts by simply changing the parameters specific to that part. Each subroutine essentially programmed as the “recipe” for a different part, containing different values for speed and positioning controls, reference positions, number of passes, and any other things pertaining to the process. The “recipe” would simply be selected by the machine operator on an HMI or some other type of control mechanism.

In this chapter we will look at three or four of the basic program flow instructions with examples on using them in actual ladder programs.

## The JSR, SBR, and RET Instructions:

The JSR instruction or “jump to subroutine”, is an action-performing instruction that will cause the program to skip to the designated subroutine SBR ladder routine. After the subroutine is performed the program returns to the rung following the JSR instruction that called it, and continues execution of the program from that point on.



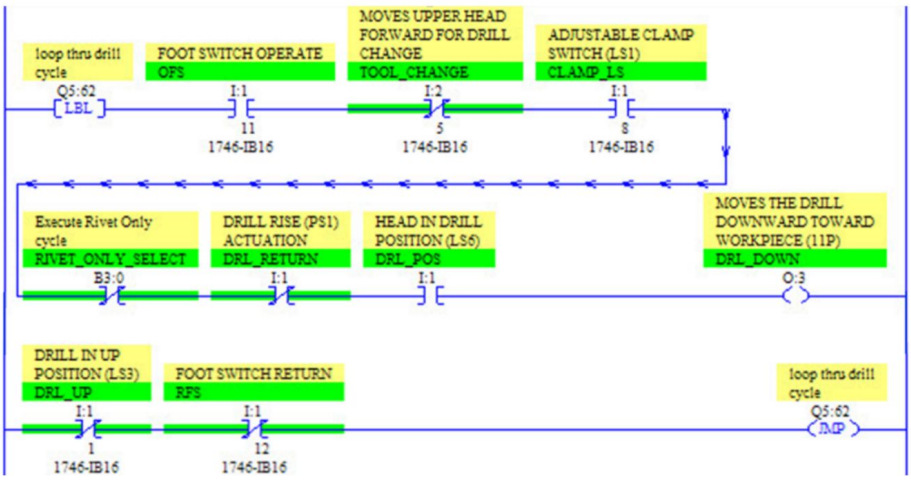

The SBR “subroutine”: This instruction serves as a unique identifier for a specific subroutine, and if used, must always be the first instruction within the desired rungs. Note that in the following example the designated subroutines are referencing individual “ladders” which are added to the project tree along the left side of the programming window. For this reason the SBR instruction is not used again in each ladder segment.

The RET “return”: designates the end of the subroutine, and causes a return to the rung that follows the JSR that

initiated the subroutine. If the RET instruction is not used, the routine execution will simply proceed to the END instruction - and then return to the rung following the JSR. The RET can be preceded by conditional instructions and is only necessary if, under certain conditions, you want to skip the rungs between the RET and the END. In other words, leave the subroutine early if certain conditions are met.

## JMP & LBL Instructions:

These instructions are used as a pair to skip portions of ladder logic – “jumping forward”, or to repeatedly execute rung segments by looping back to a specific rung of logic – “jumping backward”. The JMP will jump to the LBL designated by the same unique label number – even if in a different subroutine. The LBL must be the first instruction in the desired rung and segment of program logic.




This is an example of a “looping” type operation where it was necessary for the program to remain in a drill cycle operation until certain conditions are met. For simplicity, I am only showing the first and end rungs of this subroutine

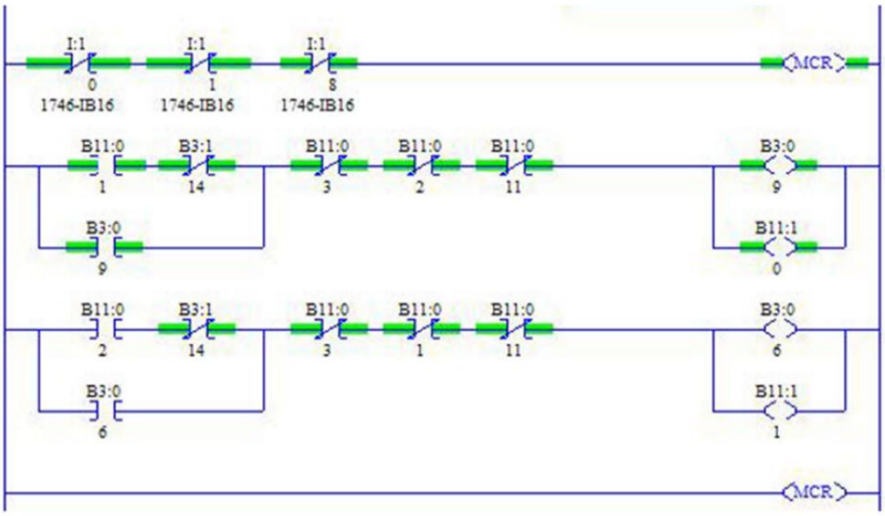
to show how the JMP and LBL can be identified and paired. This will of course, execute the program rungs in between – repeatedly - until the JMP rung becomes “false”.

## The MCR Instruction:

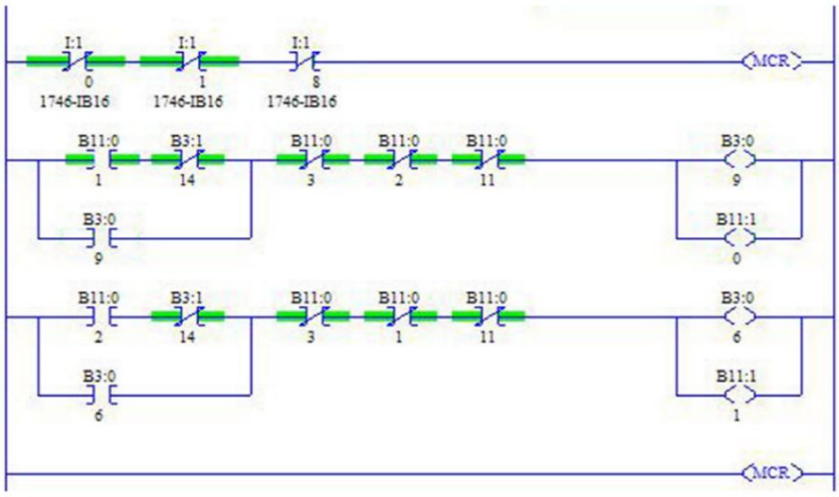
The MCR instruction, or “master control reset” is always used in pairs to define an area of ladder logic program in which you may want to disable the output instructions under certain conditions. These conditions are often related to safety, or status alarms and warnings, but should not be used as a substitution for hard-wired safe-guards such as barriers, safety mats, E-Stops and safety relays.

The defined MCR zone within the ladder logic, will clear all set outputs within the zone if they are the non-retentive type of outputs. Retentive outputs, such as a (OTL) or (OTU) - retain their current status. As you can see in the example that follows, if the rung on the initial MCR becomes “true” then all the rungs and outputs are scanned and updated as they would normally be treated. If the initial MCR rung goes “false” then the outputs within the zone defined by the MCR pair, will be set to a “false” or “low” state.

Here are examples where the initial MCR rung is “true”. Note that rungs within this zone are scanned normally – just as any other type of logic.




In this example, the initial MCR rung is “false”. Note that B3:0/9 and B11:1/0 outputs are reset to false or “low” state, even though the second rung is logically “true”.




Things to remember when using the MCR instructions are:

- Do not use any conditional logic before the ending MCR instruction. It must be on a rung by itself.

- This instruction pair is not meant to be the substitute for a “hard-wired” type of master control relay that can be utilized for an emergency stop.

- An MCR controlled zone must contain only two MCR instructions. The MCR pair encloses the rungs – a zone where outputs will be disabled under certain conditions.

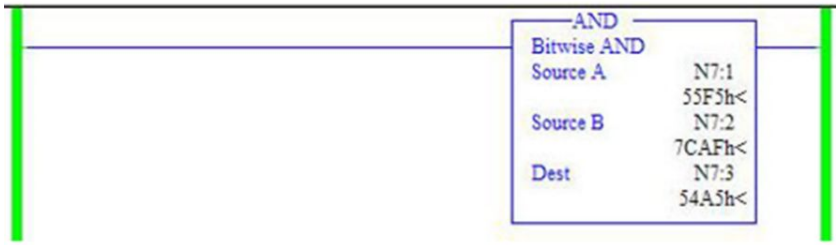
- Timers and counters will cease to operate when placed within an MCR zone – and the zone goes “false”. Although a “TOF” or “off delay” timer will activate when the rung goes “false”, just as it would normally when transitioned from a true-to-false condition.

## Logical Register Instructions

These are “action-performing” instructions that operate on registers, and then place the results in a destination register. They perform a bit-by-bit, “AND”, “OR”, “XOR”, or a “NOT” on your designated source register or word. These operations and instructions are, in general, not what you would use for setting up “logical” rung continuity in your programming. Rather, they are most often used to manipulate bits of a word into a desired form. At any rate that’s my perspective, however you may find other interesting ways to utilize these instructions in your programs. Here are examples of each.

## The AND Instruction:

Each bit of source A is “AND-ed” with the corresponding bit of source B. This simply means that both bits must be “true” or “1” for the resultant to be “true”. Results are placed into the register you designate as the destination.

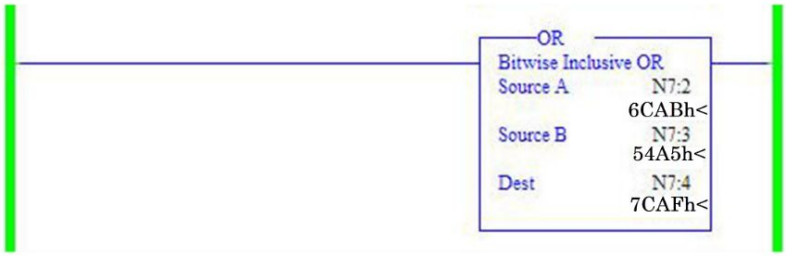



**Results of the "AND" instruction placed into Destination N7:3**

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| N7:1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
| N7:2 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| N7:3 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |

## The OR Instruction :

Each bit of source A is “OR-ed” with the corresponding bit of source B. If either condition (or both) is “true” or “1”, then the results are “true”. Results are placed into the register you designate as the destination.



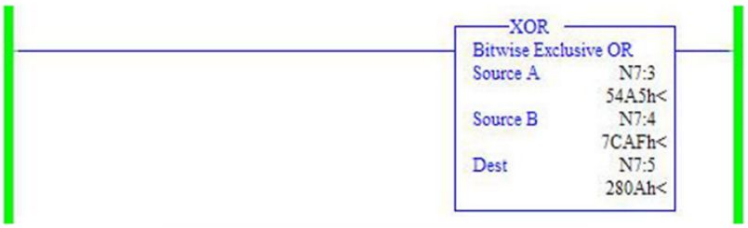

**If either the Source A or Source B bit is "true", or if both bits are "true", then the resultant bit is made "true" and put into destination N7:4**

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| N7:2 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| N7:3 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| N7:4 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |

## The XOR “exclusive or” Instruction:

Each bit of source A is “OR-ed” with the corresponding bit of source B. Remember that the “exclusive or” only allows a single condition to be “true” or “1”, rather than both. Results are placed into the register you designate as the destination.

The XOR Instruction and resultant truth-table:

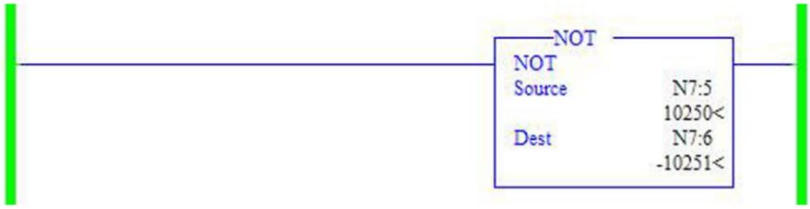



Results of the "XOR" instruction

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| N7:3 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| N7:4 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| N7:5 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |

## The NOT Instruction:

Each bit of the source word is inverted, and the results placed into the register you designate as the destination. Therefore, the results placed into the destination will be the opposite of the source bits.




**Results for the "NOT" Instruction**

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| N7:5 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
| N7:6 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |

## Concluding Comments

What I've endeavored to present in this book, are what I consider some of the more advanced concepts and instructions used in programming ladder logic. Once again, the Allen Bradley family of PLC's are very good products, in my opinion, to use when learning to program machine controls.

Hopefully, you have access to RSLogix500 software so you can develop and test your own programs. Most training seminars that teach PLC programming, provide laptops and PLC racks with I/O, and may schedule several days in the classroom. While this may be one of the best methods of learning ladder programming skills, it is not always an option to attend a three or four day class. The goal of this three book series is to convey these topics, as well as possible, from the written page - along with good illustrations and examples. I hope I've succeeded, on some level, in meeting that goal.

Some of the topics and objectives we've covered in this "Advanced Concepts" book have been:

- Setting up communication and interfacing with Allen Bradley SLC500 family of processors.
- Additional information on latching and unlatching outputs, retentive timers and counters - with examples on nesting or cascading timers and counters.
- The "math" instructions and how they can be used in scaling analog applications.

- What “scaling” is all about and other methods of achieving appropriate scaled values.

- The “comparison” instructions and how to use them in setting ranges, alarms, or to control program flow.

- The “data register” instructions and examples on how they can be used to manipulate entire words or “blocks” of word elements.

- Some of the most used “program flow control” instructions.

- The “logical register” instructions - such as the “AND”, “OR”, and “XOR”, and examples which show exactly what they do.

The 3rd book of this series, “Ladder Logic Diagnostics & Troubleshooting”, will focus on some of the tools that can be used when troubleshooting programs. These tools include a more in-depth look into the “status” data file, setting up “histograms” for inputs and outputs, and programming “bit” traps to isolate certain types of problems. Also the use of “forces” for discreet inputs and outputs, and how they differ when implemented. The final topic discussed will be some basic concepts and guidelines related to using a graphical user interface, an “HMI”, with a PLC control system.

As we began this series, I mentioned how the first PLCs, back in the 60’s were essentially built to replace older technology, the “hard-wired” relay logic systems that were then common. These types of control systems, used on machine tools, automated welding equipment, and in many other applications - were composed of groups of relays, switches and timers. The basic RSLogix 500 instruction set

we’ve discussed thus far covers much, if not all, the “functionality” of these older technologies. Hopefully you’ve realized that just about anything you want to do with programmable control systems can be accomplished, with a bit of creative thought.

Once again, PLC programming may be new to you, but if you’re competent with working on electro-mechanical and electronic components, you may enjoy doing the modifications on some older equipment by integrating a SLC 500 or Micrologix PLC. It is one of the best ways to become competent in your programming and troubleshooting skills.

As I’ve said before, I know you have many options when choosing from books and online resources that discuss these topics; so I thank you for selecting my book and hope you have benefited by the discussions covered. As always, I welcome your comments and feedback. My goal is to clearly present relevant technical topics, and I have found feedback from my readers to be an invaluable resource. If you would like to contact me with questions or comments you can do so at the following email address or website:

garyandersonbooks.com

Email: ganderson61@cox.net

Your reviews on Amazon are helpful and appreciated. If you’ve enjoyed what you’ve read and feel this book has provided a positive benefit to you, then please take a moment and write a short review.

***

Books by Gary Anderson

garyandersonbooks.com

Practical Guides for the Industrial Technician:

- Motion Control for CNC & Robotics
- Variable Frequency Drives – Installation & Troubleshooting
- Industrial Network Basics

RSLogix 500 Programming Series:

- Basics Concepts of Ladder Logic Programming
- Advanced Programming Concepts
- Diagnostics & Troubleshooting
- PID Programming Using RS Logix 500
- Program Flow Instructions Using RS Logix 500

Studio 5000 Logix Designer
A Learning Guide for ControlLogix Basics